



IES

Leonardo Da Vinci

Sovereign-Edge AI

Infraestructura Critica con **Autocura** y Defensa
mediante **IA Local**

Participantes:

Ezequiel Alpire Aranibar
Oleg Chaban Kachmaryk
Francisco Javier Lema Medina

Ciclo:

ASIR (Administración de
Sistemas Informaticos en Red)

Curso:

2025 / 2026



Índice:

- 1. Introducción: Sovereign-Edge AI..... 4
- 2. Arquitectura de red..... 4
- 3. Objetivos del Proyecto..... 5
- 3. Tecnologías Empleadas.....6
 - 3.1. Infraestructura y Virtualización..... 6
 - 3.3. Seguridad y Redes..... 8
 - 3.4. Análisis de Datos y Monitorización (Stack ELK).....8
 - 3.5. Automatización y Gestión.....9
- 4. Arquitectura del Sistema y Stack Tecnológico..... 10
- 5. Abstract.....10
- 6. Instalación de Ubuntu y configuración de red..... 11
 - 6.1. Configuración de VPN con Wireguard (Tailscale) para acceso remoto..... 14
- 7. Despliegue de Docker Engine y Docker Compose..... 18
- 8. Automatización del sistema con Ansible..... 21
 - 8.1. Creación del directorio del proyecto.....23
 - 8.2. Ejecución de las tareas definidas.....25
- 9. Despliegue de motor Ollama y modelo Llama 3.1..... 25
- 10. Interfaz Web (Open WebUI) para la IA..... 27
- 11. Acceso a Open WebUI y creación de la cuenta administrativa..... 28
 - 11.1. Configuración de la conexión entre Open WebUI y Ollama..... 29
- 12.1. Configuración de la IA para análisis de errores..... 33
- 12.2. Configuración de agentes especializados.....33
- 12.3. Despliegue e integración del entorno local..... 34
- 12.4. Validación Práctica: Resolución de Incidencias en Tiempo Real.....35
- 12.5. Cambio de motor de IA..... 36
- 13. Configuración de Honeypot Cowrie (Señuelo).....37
 - 13.1. Aislamiento del servicio SSH real..... 37
 - 13.2. Despliegue operativo del señuelo mediante Docker..... 38
 - 13.3. Configuración de Docker Compose..... 39
 - 13.4. Prueba de intrusión y validación del entorno..... 40
 - 13.5. Análisis de registros y monitorización de actividad (Logs).....41
 - 13.6. Respuesta automatizada y análisis mediante IA.....41
- 14. Hardening del sistema y Fail2Ban.....42
 - 14.1. Análisis del estado del cortafuegos (UFW)..... 42
- 15. Instalación y puesta en marcha de Fail2Ban.....43
 - 15.1. Protección de la configuración original.....43
 - 15.2. Configuración de reglas y baneo progresivo.....44
 - 15.3. Verificación de la activación de Fail2Ban.....44
 - 15.4. Prueba de funcionamiento y bloqueo de seguridad.....45
 - 15.5. Verificación de resultados y baneo de la IP.....46
 - 15.6. Análisis de registros de autenticación (auth.log).....46
 - 15.7. Desbloqueo manual de IPs.....47
 - 15.8. Análisis del log específico de Fail2Ban.....47

- 16. Instalación de Nginx Proxy Inverso..... 48
- 17. Creación de un certificado SSL autofirmado..... 49
 - 17.1. Configuración de Nginx como proxy inverso con HTTPS..... 49
 - 17.2. Verificación del estado del certificado y cifrado de la conexión..... 50
 - 17.3. Implementación de notificaciones al canal de Discord sobre intentos de acceso...51
- 18. Base de datos de logs (Elasticsearch y Filebeat) y Dashboard de Kibana:
Visualización de amenazas..... 53
 - 18.1. Configuración del fichero Docker Compose.....53
 - 18.2. Puesta en marcha de la infraestructura de análisis mediante contenedores.....54
 - 18.3. Acceso a la interfaz de gestión y visualización en Kibana.....55
 - 18.4. Configuración de alias en Kibana.....56
 - 18.5. Monitorización activa y análisis de registros en tiempo real..... 56
 - 18.6. Configuración del contenedor de Filebeat.....57
 - 18.7. Configuración de visualizaciones y analítica en Kibana Lens.....57
 - 18.8. Personalización de métricas y filtros en Kibana Lens..... 58
 - 18.9. Selección de modelos de representación visual en Kibana.....59
 - 18.10. Gestión de rangos temporales para el análisis de incidentes..... 60
 - 18.11. Consolidación del Dashboard de seguridad en Kibana.....60
 - 18.12. Almacenamiento del Dashboard de análisis..... 62
- 19. Conclusiones.....63
- 20. Posibles aplicaciones y mejoras..... 63
- 21. Anexos del Proyecto.....64
- 22. Bibliografía..... 66

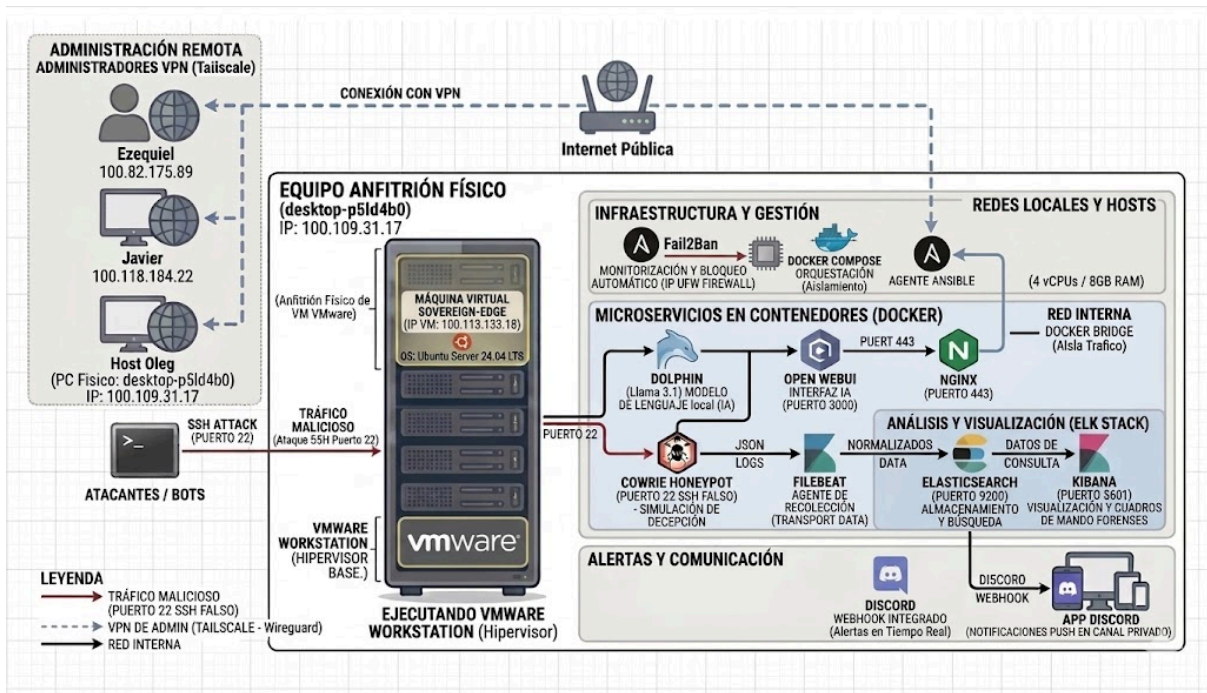
1. Introducción: Sovereign-Edge AI.

En el mundo tecnológico actual, la centralización de servicios en la nube plantea dificultades críticas sobre todo en términos de privacidad y soberanía de los datos. El proyecto **Sovereign-Edge AI** surge como una respuesta a este problema, proponiendo un diseño para funcionar de forma autónoma y segura en una red local.

La arquitectura del proyecto se fundamenta en la unión de dos áreas clave: la Inteligencia Artificial y la ciberseguridad. Por un lado, se implementa un modelo de lenguaje de gran tamaño (LLM) que permite el procesamiento de información sensible y crítica sin dependencia de terceros. Por otro lado, el sistema utiliza una infraestructura de defensa activa basada en técnicas de engaño o **Honeypots**. De esta manera, el servidor no solo ofrece apoyo sino que actúa como un sensor de amenazas, capturando datos sobre ataques externos para fortalecer y mejorar su propia configuración y así prepararse para posibles futuros ataques.

El uso de tecnologías como la configuración en contenedores con **Docker**, la automatización con **Ansible** y el análisis de datos con el **Stack ELK**, garantiza que sea una solución profesional, escalable y resistente ante posibles ataques.

2. Arquitectura de red



3. Objetivos del Proyecto.

Los objetivos de este proyecto se centran en el despliegue de un equipo local que sea capaz de autogestionarse y protegerse de forma autónoma. Se busca que la infraestructura no solo sea funcional, sino que sea robusta frente a ataques y respetuosa con la privacidad de los datos. Se detallan a continuación:

1. **Automatización del despliegue:** Reducir la intervención manual mediante el uso de **Ansible** para que la configuración del sistema, la instalación de paquetes y el mantenimiento sean procesos rápidos, documentados y libres de errores humanos.
2. **Soberanía de datos mediante IA local:** Implementar un modelo de lenguaje funcional (**Dolphin**) que trabaje en el hardware local. El objetivo es demostrar que se pueden obtener servicios de IA avanzados sin que la información confidencial salga de la red privada.
3. **Implementación de defensa activa:** Configurar un entorno de engaño (**Honeypot**) que actúe como primera línea de defensa. El objetivo es desviar el tráfico malicioso hacia un entorno controlado para estudiar el comportamiento de los atacantes sin poner en riesgo los datos reales.
4. **Orquestación modular con Docker:** Utilizar contenedores para que cada servicio como IA, base de datos o seguridad funcione de forma aislada. Esto permite que si un servicio falla o necesita actualizarse, el resto del servidor siga operativo mejorando la disponibilidad del servidor.
5. **Blindaje y respuesta automática:** Establecer reglas de firewall y sistemas de bloqueo automático con **Fail2Ban** para que el servidor aprenda a defenderse solo. Si se detecta un patrón de ataque, el sistema debe ser capaz de cortar el acceso al intruso de forma inmediata.
6. **Análisis y monitorización forense:** Centralizar todos los registros de seguridad en un stack **ELK** para convertir los logs “infinitos” en gráficos comprensibles. Se busca identificar de dónde vienen los ataques y qué métodos utilizan para intentar comprometer el sistema.
7. **Acceso remoto seguro:** Garantizar que la administración del servidor se realice a través de una red privada cifrada con **Tailscale**. El objetivo es poder gestionar el servidor desde cualquier lugar sin tener que exponer puertos vulnerables.
8. **Escalabilidad:** Asegurar que todo el entorno pueda replicarse en un nuevo servidor en cuestión de minutos. Gracias al uso de código para crear la infraestructura, el proyecto no depende de una única máquina sino que es fácilmente escalable.

En conjunto, estos objetivos pretenden demostrar que es factible gestionar una infraestructura de **Edge Computing** que combine la potencia de la Inteligencia Artificial con altos niveles de seguridad y privacidad, adaptándose a las necesidades de una organización profesional.

3. Tecnologías Empleadas.

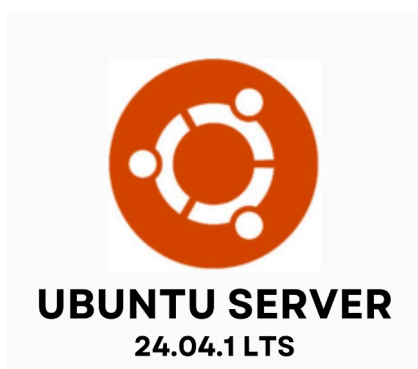
Para construir la infraestructura de Sovereign-Edge AI que sea a la vez potente y segura, se ha seleccionado un stack tecnológico que permite el aislamiento de procesos y el aprovechamiento máximo del hardware.

3.1. Infraestructura y Virtualización.

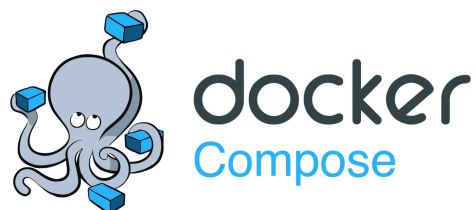
- **VMware Workstation:** Actúa como el hipervisor base. Se ha configurado una máquina virtual con **4 núcleos de CPU y 8 GB de RAM**, una asignación equilibrada para permitir que el modelo de lenguaje (IA) procese datos sin bloquear los servicios de seguridad.



- **Ubuntu Server 24.04 LTS:** Es el núcleo del proyecto. Se eligió esta distribución por su estabilidad y su excelente soporte para entornos de contenedores. Durante la instalación, se realizó un particionado manual para garantizar un acceso más directo al disco y facilitar la gestión de volúmenes de datos.



- **Docker & Docker Compose:** Son las herramientas de orquestación. En lugar de instalar servicios directamente en el sistema, se utilizan contenedores. Esto permite que la IA, el Honeypot y la base de datos vivan en entornos aislados, compartiendo recursos pero sin interferir entre sí, lo que facilita enormemente las actualizaciones y la portabilidad de cada uno de ellos.



3.2. Inteligencia Artificial y Capa Visual.

- **Ollama:** Es el motor encargado de ejecutar los Modelos de Lenguaje de Gran Tamaño de forma local. Permite cargar modelos como **Llama 3.1** o **Dolphin** que se han usado en este proyecto, gestionando la descarga y la ejecución eficiente sobre la CPU/RAM del servidor.



- **Open WebUI:** Proporciona la interfaz gráfica para interactuar con la IA. Se despliega mediante Docker y se conecta internamente con Ollama, ofreciendo una experiencia similar a soluciones en la nube pero manteniendo todo el tráfico de datos dentro del nodo local.



3.3. Seguridad y Redes.

- **Cowrie Honeypot:** Es la pieza central de la defensa activa. Emula un servicio SSH falso en el puerto 22 que registra cada intento de inicio de sesión, capturando los nombres de usuario, contraseñas y comandos que los atacantes intentan ejecutar.



HoneyPot

- **Tailscale (VPN):** Implementa una red privada basada en el protocolo **Wireguard**. Gracias a esto, el servidor tiene una IP fija privada que permite gestionarlo por SSH desde cualquier lugar de forma cifrada, eliminando la necesidad de abrir puertos vulnerables en el router que conecta a nuestra maquina real.



- **Fail2Ban:** Actúa como un sistema de respuesta automática. Analiza los logs en tiempo real y, si detecta múltiples fallos de acceso en el SSH real (puerto 2222), bloquea la IP del atacante mediante el cortafuegos UFW.



FAIL2BAN

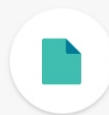
3.4. Análisis de Datos y Monitorización (Stack ELK)

- **Elasticsearch:** Funciona como el motor de almacenamiento y búsqueda. Es una base de datos diseñada para manejar grandes volúmenes de logs de forma indexada y rápida.



elasticsearch

- **Filebeat:** Es el agente de transporte. Se encarga de leer los archivos JSON generados por el HoneyPot y enviarlos de forma estructurada a la base de datos.



Filebeat

- **Kibana:** Es la herramienta de visualización. Permite crear cuadros de mando interactivos, donde los datos de los ataques se transforman en mapas de calor, gráficos de tarta y tablas de estadísticas, también nos permite visualizar los logs en tiempo real con gráficos estadísticos sobre los accesos.



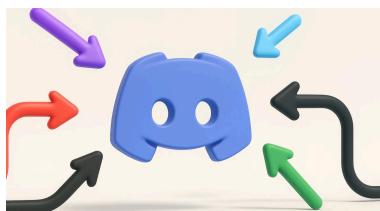
3.5. Automatización y Gestión

- **Ansible:** Se utiliza para la gestión inmutable del servidor. A través de archivos de configuración (*Playbooks*), se automatizan tareas como la actualización del sistema, la configuración de reglas de firewall o el despliegue de nuevos contenedores desde una máquina de control externa.



ANSIBLE

- **Discord Webhooks:** Integrados para el sistema de alertas. El servidor envía mensajes automáticos a un canal de Discord privado cada vez que el Honeypot detecta actividad, permitiendo una monitorización constante de los accesos y alertas de los mismos con los diferentes parámetros de accesos ip, fecha, comandos ejecutados, etc.



4. Arquitectura del Sistema y Stack Tecnológico.

Para el desarrollo del proyecto utilizaremos herramientas alineadas con los módulos del ciclo ASIR:

Área	Tecnologías Implementadas
Sistemas	Servidores basados en Linux (Debian/Ubuntu) . Orquestación de servicios mediante Docker y Docker Compose para asegurar la inmutabilidad.
Automatización	Uso de Ansible para la gestión de "Infraestructura como Código" (IaC) y recuperación rápida de configuraciones.
Seguridad y Redes	Cowrie como <i>honeypot</i> de SSH/Telnet. Gestión de tráfico y certificados SSL con Nginx Proxy Manager . Control de tráfico mediante firewalls dinámicos vinculados a la IA.
IA y Datos	Motor Ollama (Llama 3.1) ejecutado en local. Stack ELK (Elasticsearch y Kibana) para el almacenamiento masivo de <i>logs</i> y creación de <i>dashboards</i> de seguridad.

5. Abstract.

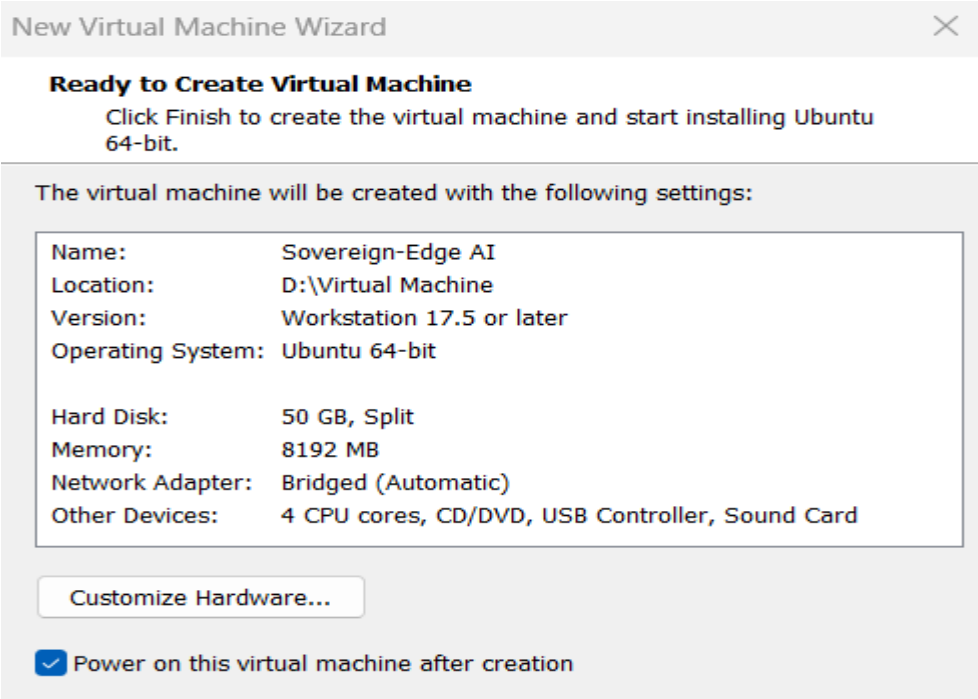
The idea behind Sovereign-Edge AI is to build a server that can look after itself. We made this for small places like local clinics or NGOs that don't have a tech expert available all the time. We wanted a system that stays running even if there's a technical problem or a hacking attempt.

To make this work, we used Docker and Ansible. These tools allow the server to fix itself by restarting any broken service in seconds. For security, we set up a honeypot which is basically a trap to distract hackers and a VPN that we can connect safely from home.

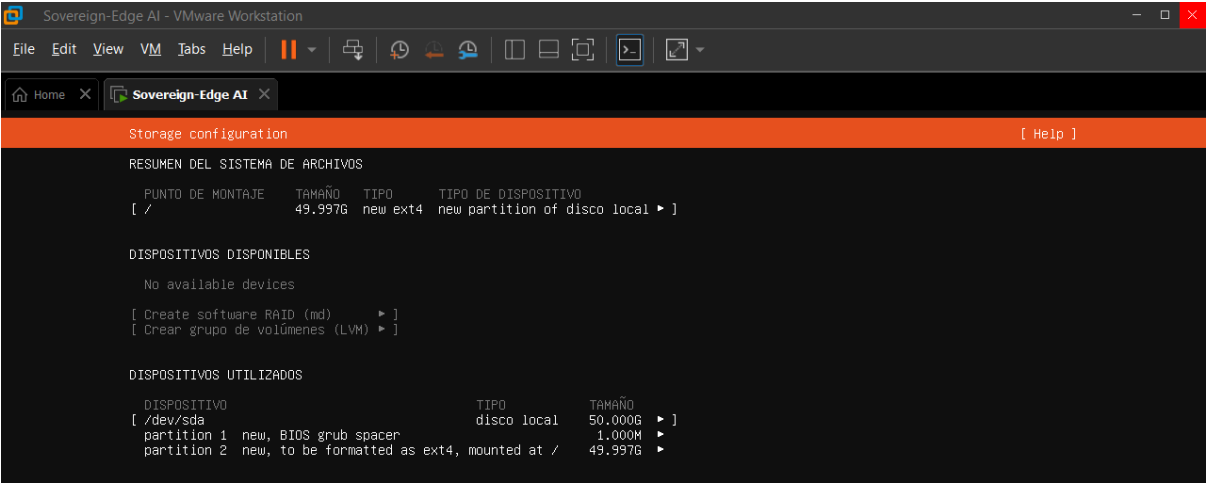
The best part is the local AI (Dolphin). Since it runs inside the server without internet, it keeps all data private. This AI helps us read logs and block bad users automatically. We also added a dashboard using Kibana to see exactly what's happening with our security in real-time. It's a simple, smart and private way to keep a server safe and always online.

6. Instalación de Ubuntu y configuración de red.

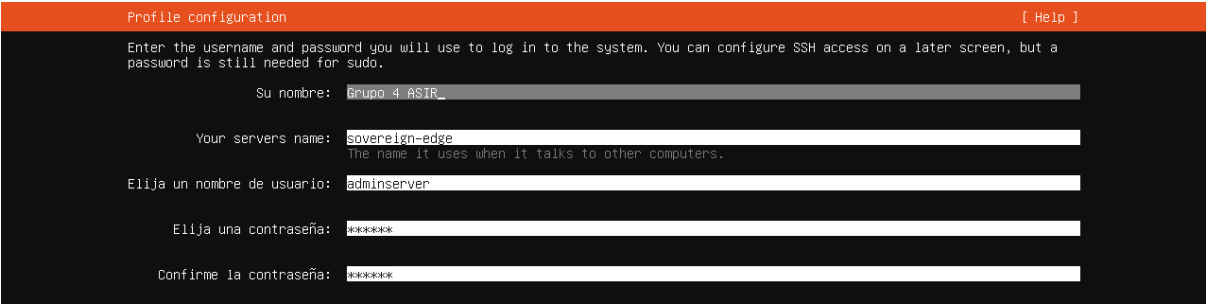
Aquí se ve que le he puesto 50 GB de disco duro y 8GB de memoria RAM para que todo vaya rápido. También configuré la red en modo puente para que la máquina funcione de forma independiente



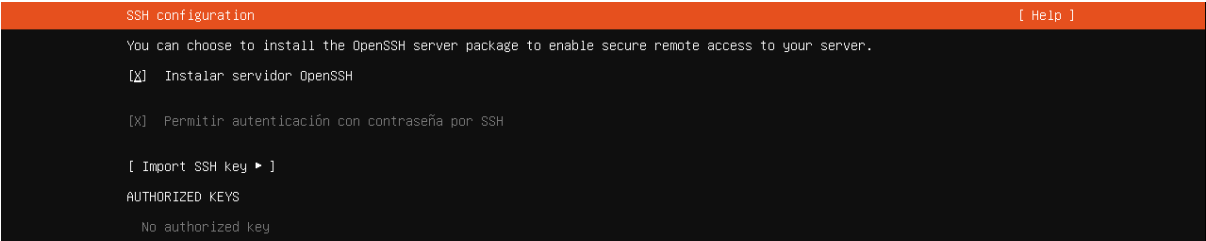
Se puede ver cómo el sistema reconoce las particiones del disco duro virtual de 50 GB y prepara el formato ext4 en la raíz para dejar listo el espacio donde se guardará todo el sistema operativo y las herramientas del proyecto



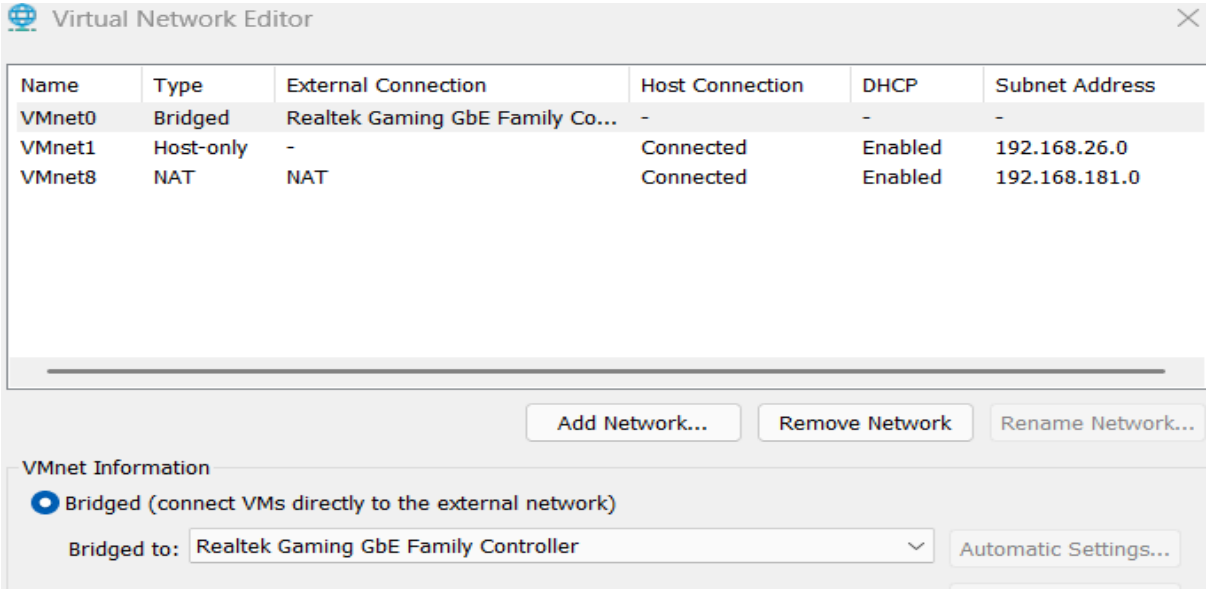
Defino el nombre del equipo como sovereign-edge y creo la cuenta de administrador con el usuario adminserver y su contraseña para poder acceder y gestionar el servidor de forma segura una vez que termine la instalación



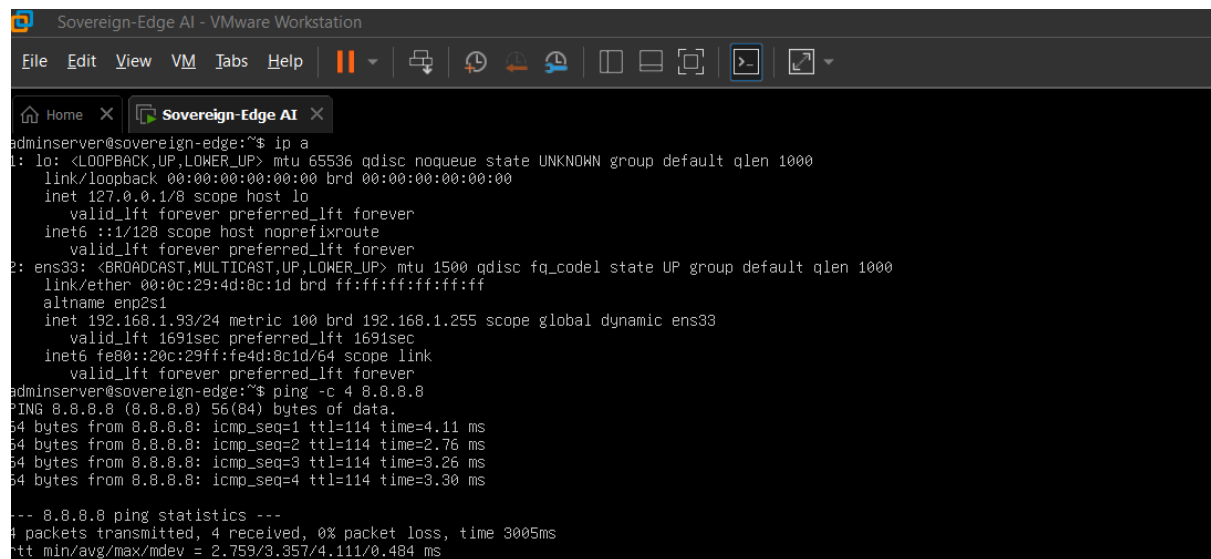
Es la pantalla donde marco la casilla para instalar el servidor OpenSSH en Ubuntu para conectarme remotamente



Se ve cómo tengo seleccionado el modo puente (Bridged) en la tarjeta de red física del ordenador para conectarla directamente con la máquina virtual, asegurando que reciba su propia dirección IP y funcione como un dispositivo real más dentro de la red.



Uso el comando ip a para ver la dirección IP que le ha asignado mi router y luego hago un ping a los servidores de Google (8.8.8.8) para asegurar que tiene acceso a internet y responde correctamente sin perder datos



```
adminserver@sovereign-edge:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:4d:8c:1d brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.1.93/24 metric 100 brd 192.168.1.255 scope global dynamic ens33
        valid_lft 1691sec preferred_lft 1691sec
    inet6 fe80::20c:29ff:fe4d:8c1d/64 scope link
        valid_lft forever preferred_lft forever
adminserver@sovereign-edge:~$ ping -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data:
64 bytes from 8.8.8.8: icmp_seq=1 ttl=114 time=4.11 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=114 time=2.76 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=114 time=3.26 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=114 time=3.30 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 2.759/3.357/4.111/0.484 ms
```

6.1. Configuración de VPN con Wireguard (Tailscale) para acceso remoto.

Despliegue de Tailscale para la conexión mediante VPN hacia la diferente infraestructura a crear posteriormente en las semanas siguientes:

```
adminserver@sovereign-edge:~$ curl -fsSL https://tailscale.com/install.sh | sh
Installing Tailscale for ubuntu noble, using method apt
+ sudo mkdir -p --mode=0755 /usr/share/keyrings
+ curl -fsSL https://pkgs.tailscale.com/stable/ubuntu/noble.noarmor.gpg
+ sudo tee /usr/share/keyrings/tailscale-archive-keyring.gpg
+ sudo chmod 0644 /usr/share/keyrings/tailscale-archive-keyring.gpg
+ curl -fsSL https://pkgs.tailscale.com/stable/ubuntu/noble.tailscale-keyring.list
+ sudo tee /etc/apt/sources.list.d/tailscale.list
# Tailscale packages for ubuntu noble
deb [signed-by=/usr/share/keyrings/tailscale-archive-keyring.gpg] https://pkgs.tailscale.com/stable/ubuntu noble main
+ sudo chmod 0644 /etc/apt/sources.list.d/tailscale.list
+ sudo apt-get update
Obj:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Obj:2 http://es.archive.ubuntu.com/ubuntu noble InRelease
Obj:3 http://es.archive.ubuntu.com/ubuntu noble-updates InRelease
Obj:4 http://es.archive.ubuntu.com/ubuntu noble-backports InRelease
Des:5 https://pkgs.tailscale.com/stable/ubuntu noble InRelease
Descargados 6.581 B en 1s (12,4 kB/s)
Leyendo lista de paquetes... Hecho
+ [ -n ]
+ sudo apt-get install -y tailscale tailscale-archive-keyring
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
tailscale ya está en su versión más reciente (1.94.2).
tailscale-archive-keyring ya está en su versión más reciente (1.35.181).
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 6 no actualizados.
+ [ false = true ]
+ set +x
Installation complete! Log in to start using Tailscale by running:
sudo tailscale up
```

En Tailscale procedemos a realizar conexiones con nuestros equipos hacia la red virtual, donde se encuentran nuestros equipos de proyecto:



ezequielalp21@gmail.com 

Connect device

You are about to connect the device **sovereign-edge** to the **ezequielalp21@gmail.com** tailnet.

Connect

▼ Device details

Public key	nodekey:f93bd0746c79d322cf623167c329...
Hostname	sovereign-edge
Operating system	linux (6.8.0-101-generic)
Tailscale version	1.94.2-t0a29cf18b-g3f044c9f6

Levantamos la interfaz de Tailscale para establecer más adelante la conexión:



```
adminserver@sovereign-edge:~$ sudo tailscale up
To authenticate, visit:
    https://login.tailscale.com/a/9cdabc7015699
Success.
adminserver@sovereign-edge:~$
```

Podemos observar la ip del túnel de nuestro servidor que nos proporciona Tailscale para la conexión:



```
adminserver@sovereign-edge:~$ tailscale ip -4
100.113.133.18
adminserver@sovereign-edge:~$ _
```

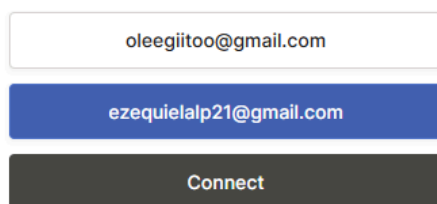
100.113.133.18

Acceso habilitado a los diferentes equipos físicos hacia el servidor Ubuntu.

Connect device

You are about to connect the device **DESKTOP-P5LD4B0**
This may give the device access to other resources in the
selected tailnet, based on ACLs.

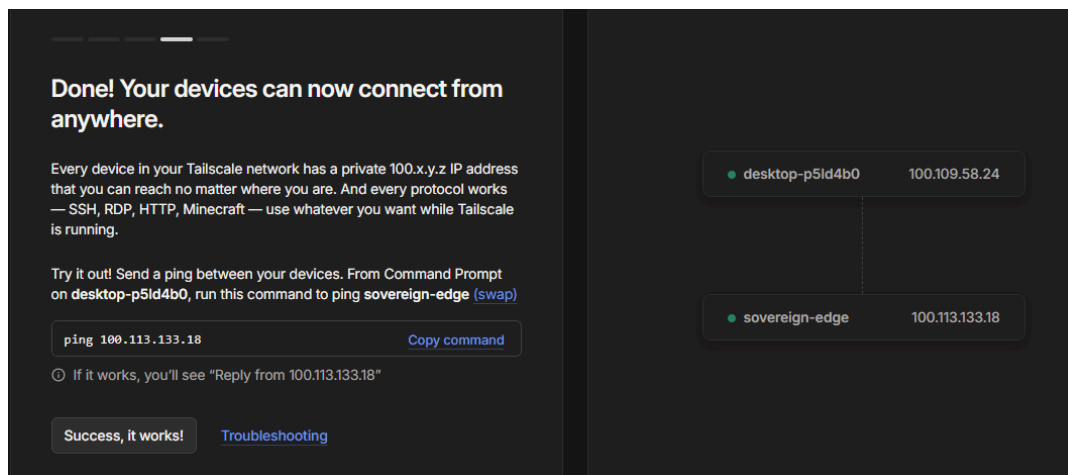
Select a tailnet



oleegiitoo@gmail.com

ezequielalp21@gmail.com

Connect



Conectividad realizada entre el servidor y uno de los equipos fisicos:

```
C:\Users\Oleg>ping 100.113.133.18

Haciendo ping a 100.113.133.18 con 32 bytes de datos:
Respuesta desde 100.113.133.18: bytes=32 tiempo=19ms TTL=64
Respuesta desde 100.113.133.18: bytes=32 tiempo=9ms TTL=64
Respuesta desde 100.113.133.18: bytes=32 tiempo=8ms TTL=64
Respuesta desde 100.113.133.18: bytes=32 tiempo=9ms TTL=64

Estadísticas de ping para 100.113.133.18:
    Paquetes: enviados = 4, recibidos = 4, perdidos = 0
    (0% perdidos),
    Tiempos aproximados de ida y vuelta en milisegundos:
        Mínimo = 8ms, Máximo = 19ms, Media = 11ms
```


Obtenemos acceso al servidor mediante ssh gracias a la vpn Tailscale

```
C:\Users\Oleg>ssh adminserver@100.113.133.18
The authenticity of host '100.113.133.18 (100.113.133.18)' can't be established.
ED25519 key fingerprint is SHA256:5m/vuo/njoLyIUHjgijViZhFlhJq1lCFupEQdeKqZKI.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '100.113.133.18' (ED25519) to the list of known hosts.
adminserver@100.113.133.18's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-101-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of mar 10 mar 2026 22:53:26 UTC

System load:  0.0               Processes:            229
Usage of /:   14.7% of 48.91GB   Users logged in:     1
Memory usage: 5%               IPv4 address for ens33: 192.168.1.145
Swap usage:   0%

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

El mantenimiento de seguridad expandido para Applications está desactivado

Se pueden aplicar 0 actualizaciones de forma inmediata.

Active ESM Apps para recibir futuras actualizaciones de seguridad adicionales.
Vea https://ubuntu.com/esm o ejecute «sudo pro status»

Last login: Tue Mar 10 21:38:43 2026 from 100.82.175.89
adminserver@sovereign-edge:~$
```

7. Despliegue de Docker Engine y Docker Compose.

En el terminal donde descargo e instalo los certificados de seguridad de Ubuntu usando el comando `sudo apt-get install --reinstall ca-certificates`, hago esto para solucionar un fallo con los certificados del sistema y asegurarme de que la máquina pueda conectar de forma segura a páginas web y servidores externos sin dar errores de confianza

```
adminserver@sovereign-edge:~$ sudo apt update
[sudo] password for adminserver:
Obj:1 http://es.archive.ubuntu.com/ubuntu noble InRelease
Obj:2 http://es.archive.ubuntu.com/ubuntu noble-updates InRelease
Obj:3 http://es.archive.ubuntu.com/ubuntu noble-backports InRelease
Obj:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Des:5 https://pkgs.tailscale.com/stable/ubuntu noble InRelease
Descargados 6.581 B en 0s (14,3 kB/s)
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se pueden actualizar 2 paquetes. Ejecute «apt list --upgradable» para verlos.
adminserver@sovereign-edge:~$ sudo apt install ca-certificates curl gnupg lsb-release -y
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
ca-certificates ya está en su versión más reciente (20240203).
fijado ca-certificates como instalado manualmente.
curl ya está en su versión más reciente (8.5.0-2ubuntu10.7).
fijado curl como instalado manualmente.
gnupg ya está en su versión más reciente (2.4.4-2ubuntu17.4).
fijado gnupg como instalado manualmente.
lsb-release ya está en su versión más reciente (12.0-2).
fijado lsb-release como instalado manualmente.
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 2 no actualizados.
adminserver@sovereign-edge:~$
```

Creación del directorio para el despliegue de Docker.

```
adminserver@sovereign-edge:~$ sudo mkdir -p /etc/apt/keyrings
adminserver@sovereign-edge:~$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
adminserver@sovereign-edge:~$
```

Comando para el despliegue y funcionamiento de nuestro contenedor de Docker, con sus respectivos parámetros.

```
adminserver@sovereign-edge:~$ echo \
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker
.gpg] https://download.docker.com/linux/ubuntu \
$(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list
> /dev/null
adminserver@sovereign-edge:~$
```

Es la pantalla de la terminal donde instalo Docker y sus componentes necesarios en el servidor usando el comando `sudo apt install docker-ce docker-ce-cli containerd.io docker-compose-plugin -y`. Hago esto para preparar el sistema y poder levantar más adelante los contenedores aislados del Honeypot y de la interfaz de Inteligencia Artificial.

```
adminserver@sovereign-edge:~$ sudo apt install docker-ce docker-ce-cli containerd.io docker-compose-plugin -y
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes adicionales:
  docker-buildx-plugin docker-ce-rootless-extras libslirp0 pigz
  slirp4netns
Paquetes sugeridos:
  cgroupfs-mount | cgroup-lite docker-model-plugin
Se instalarán los siguientes paquetes NUEVOS:
  containerd.io docker-buildx-plugin docker-ce docker-ce-cli
  docker-ce-rootless-extras docker-compose-plugin libslirp0 pigz
  slirp4netns
0 actualizados, 9 nuevos se instalarán, 0 para eliminar y 2 no actualizados.
Se necesita descargar 97,0 MB de archivos.
Se utilizarán 392 MB de espacio de disco adicional después de esta operación.
Des:1 https://download.docker.com/linux/ubuntu noble/stable amd64 containerd.io amd64 2.2.1-1~ubuntu.24.04~noble [23,4 MB]
Des:2 http://es.archive.ubuntu.com/ubuntu noble/universe amd64 pigz amd64 2.8-1 [65,6 kB]
Des:3 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-ce-cli amd64 5:29.3.0-1~ubuntu.24.04~noble [16,4 MB]
Des:4 http://es.archive.ubuntu.com/ubuntu noble/main amd64 libslirp0 amd64 4.7.0-1ubuntu3 [63,8 kB]
Des:5 http://es.archive.ubuntu.com/ubuntu noble/universe amd64 slirp4netns amd64 1.2.1-1build2 [34,9 kB]
Des:6 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-ce amd64 5:29.3.0-1~ubuntu.24.04~noble [22,6 MB]
```

Verificación de la versión de Docker:

```
adminserver@sovereign-edge:~$ docker compose version
Docker Compose version v5.1.0
adminserver@sovereign-edge:~$ sudo usermod -aG docker adminserver
adminserver@sovereign-edge:~$
```

Habilitamos el servicio de Docker:

```
adminserver@sovereign-edge:~$ sudo systemctl enable docker.service
Synchronizing state of docker.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable docker
adminserver@sovereign-edge:~$ sudo systemctl enable containerd.service
adminserver@sovereign-edge:~$ sudo systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-03-10 22:16:03 UTC; 19min ago
 TriggeredBy: ● docker.socket
           Docs: https://docs.docker.com
        Main PID: 2543 (dockerd)
           Tasks: 10
          Memory: 27.4M (peak: 28.8M)
             CPU: 362ms
          CGroup: /system.slice/docker.service
                  └─2543 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
```

En el terminal instalo y activo la herramienta bash-completion mediante los comandos `sudo apt install bash-completion -y` y `source /usr/share/bash-completion/bash_completion`, esto para habilitar el autocompletado inteligente de comandos con la tecla tab, lo que me permite trabajar mucho más rápido y sin errores.

```
adminserver@sovereign-edge:~$ sudo apt install bash-completion -y
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
bash-completion ya está en su versión más reciente (1:2.11-8).
fijado bash-completion como instalado manualmente.
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 2 no actualizados.
adminserver@sovereign-edge:~$ source /usr/share/bash-completion/bash_completion
adminserver@sovereign-edge:~$
```

Despliegue del contenedor de Docker:

```
adminserver@sovereign-edge:~$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
17eec7bbc9d7: Pull complete
ea52d2000f90: Download complete
Digest: sha256:ef54e839ef541993b4e87f25e752f7cf4238fa55f017957c2eb44077083d7a6a
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

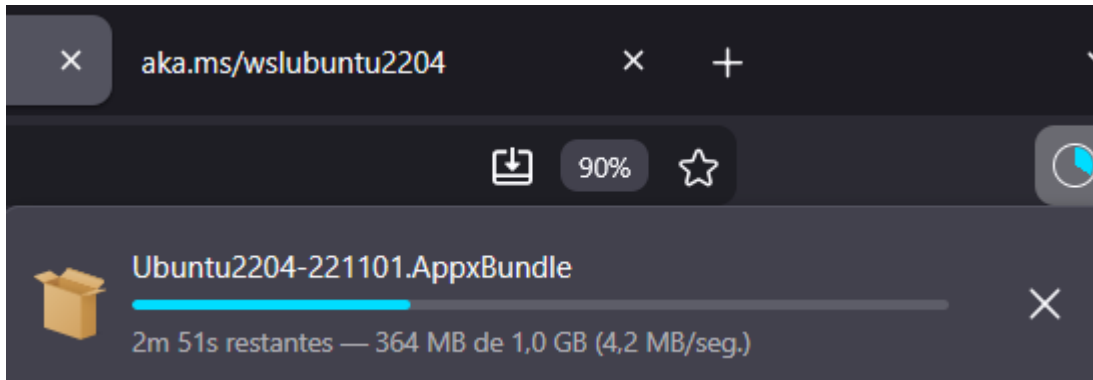
To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

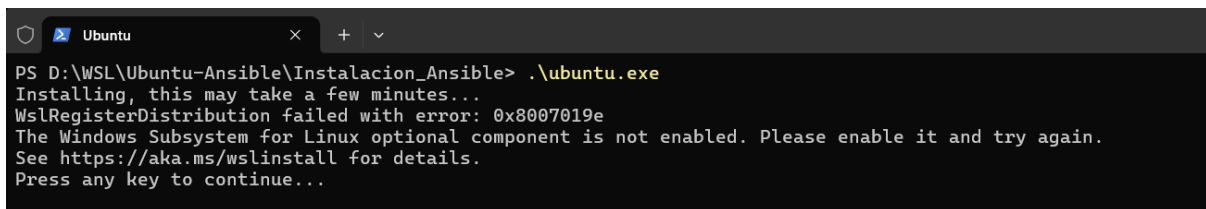
For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

8. Automatización del sistema con Ansible.

Procedemos a realizar la instalación de Ansible

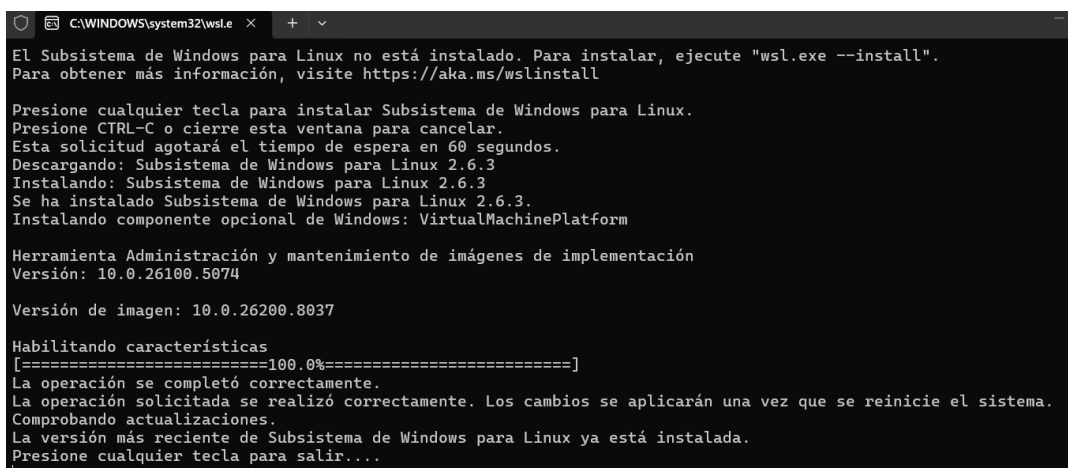


Configuración del entorno de control WSL + Ubuntu en Windows 11



Se procede a realizar la instalación:

En la imagen analizada se observa el proceso de instalación y configuración del Subsistema de Windows para Linux (WSL, Windows Subsystem for Linux) mediante la consola de comandos de Windows (CMD). WSL es una tecnología proporcionada por Microsoft que permite ejecutar distribuciones de Linux de forma nativa sobre sistemas Windows, sin necesidad de utilizar máquinas virtuales tradicionales o configuración compleja de arranque dual.



En la imagen se observa la primera ejecución de una distribución Linux (Ubuntu) instalada previamente dentro del Subsistema de Windows para Linux (WSL). Este paso se realiza tras haber completado correctamente la instalación de WSL y haber reiniciado el sistema operativo. También nos procede a dar la bienvenida a nuestro sistema operativo en este caso de Ubuntu.

```
ezequiel@DESKTOP-3NM8BMV x + v
PS D:\WSL\Ubuntu-Ansible\Instalacion_Ansible> .\ubuntu.exe
Installing, this may take a few minutes...
Please create a default UNIX user account. The username does not need to match your Windows username.
For more information visit: https://aka.ms/wslusers
Enter new UNIX username: ezequiel
New password:
Retype new password:
passwd: password updated successfully
Installation successful!
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

Welcome to Ubuntu 22.04.1 LTS (GNU/Linux 6.6.87.2-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

This message is shown once a day. To disable it please create the
/home/ezequiel/.hushlogin file.
ezequiel@DESKTOP-3NM8BMV:~$
```



En la imagen se muestra el proceso de instalación de la herramienta Ansible dentro de una distribución Ubuntu ejecutándose sobre el Subsistema de Windows para Linux (WSL). Ansible es una plataforma de automatización ampliamente utilizada para la gestión de configuraciones, despliegue de aplicaciones y orquestación de sistemas.

```
ezequiel@DESKTOP-3NM8BMV:~/proyecto-ansible$ sudo apt install ansible -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  ieee-data python3-argcomplete python3-distutils python3-dnspython python3-lib2to3 python3-libcloud python3-lockfile python3-netaddr python3-pycryptodome
  python3-requests-toolbelt python3-selinux python3-simplejson
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  ansible-core
The following NEW packages will be installed:
  ansible ansible-core
0 upgraded, 2 newly installed, 0 to remove and 207 not upgraded.
Need to get 0 B/18.9 MB of archives.
After this operation, 211 MB of additional disk space will be used.
Selecting previously unselected package ansible-core.
(Reading database ... 29282 files and directories currently installed.)
Preparing to unpack .../ansible-core-2.17.14-1ppa-jammy_all.deb ...
Unpacking ansible-core (2.17.14-1ppa-jammy) ...
Selecting previously unselected package ansible.
Preparing to unpack .../ansible-10.7.0-1ppa-jammy_all.deb ...
Unpacking ansible (10.7.0-1ppa-jammy) ...
Setting up ansible-core (2.17.14-1ppa-jammy) ...
Setting up ansible (10.7.0-1ppa-jammy) ...
Processing triggers for man-db (2.10.2-1) ...
ezequiel@DESKTOP-3NM8BMV:~/proyecto-ansible$ ansible --version
ansible [core 2.17.14]
  config file = /etc/ansible/ansible.cfg
  configured module search path = ['/home/ezequiel/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /home/ezequiel/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.10.6 (main, Aug 10 2022, 11:40:04) [GCC 11.3.0] (/usr/bin/python3)
  jinja version = 3.0.3
  libyaml = True
```


8.1. Creación del directorio del proyecto.

Este comando crea un nuevo directorio llamado proyecto-ansible, el cual se utilizará como carpeta raíz del proyecto Ansible. Centralizar los archivos del proyecto en un directorio específico facilita la organización y el mantenimiento de la infraestructura automatizada.

```
ezequiel@DESKTOP-3NM8MV:~$ mkdir proyecto-ansible
ezequiel@DESKTOP-3NM8MV:~$ cd proyecto-ansible
ezequiel@DESKTOP-3NM8MV:~/proyecto-ansible$ nano hosts.ini
ezequiel@DESKTOP-3NM8MV:~/proyecto-ansible$ cat hosts.ini
[servidores]
servidor_edge ansible_host=100.113.133.18 ansible_user=adminserver
ezequiel@DESKTOP-3NM8MV:~/proyecto-ansible$
```

En la imagen se muestra la ejecución de una prueba de conectividad entre Ansible y el servidor remoto definido previamente en el archivo de inventario. Esta comprobación es un paso fundamental antes de comenzar a ejecutar playbooks, ya que permite verificar que Ansible puede comunicarse correctamente con los nodos gestionados.

```
ezequiel@DESKTOP-3NM8MV:~/proyecto-ansible$ ansible servidores -m ping -i hosts.ini --ask-pass
SSH password:
[WARNING]: Platform linux on host servidor_edge is using the discovered Python interpreter at /usr/bin/python3.12, but future installation of another
Python interpreter could change the meaning of that path. See https://docs.ansible.com/ansible-core/2.17/reference_appendices/interpreter_discovery.html
for more information.
servidor_edge | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.12"
  },
  "changed": false,
  "ping": "pong"
}
ezequiel@DESKTOP-3NM8MV:~/proyecto-ansible$
```

En la imagen se muestra el contenido del archivo mantenimiento.yml, que corresponde a un playbook de Ansible diseñado para realizar tareas de mantenimiento automático de nuestro servidor.

```
ezequiel@DESKTOP-3NM8MV:~/proyecto-ansible$ cat mantenimiento.yml
- name: Mantenimiento automatico del servidor
  hosts: servidores
  become: yes

  tasks:
    - name: Actualizar la lista de programas
      apt:
        update_cache: yes

    - name: Actualizar todo el sistema
      apt:
        upgrade: dist

    - name: Limpiar basura y paquetes viejos
      apt:
        autoremove: yes

    - name: Limpiar el archivo temporal de descargas
      apt:
        autoclean: yes
ezequiel@DESKTOP-3NM8MV:~/proyecto-ansible$
```

 mantenimiento.yml

En la imagen se muestra la ejecución del playbook mantenimiento.yml mediante Ansible, con el objetivo de aplicar tareas de mantenimiento automático sobre el servidor remoto definido en el inventario. Este paso representa la puesta en práctica de la automatización previamente configurada.

Los parámetros utilizados tienen la siguiente función:

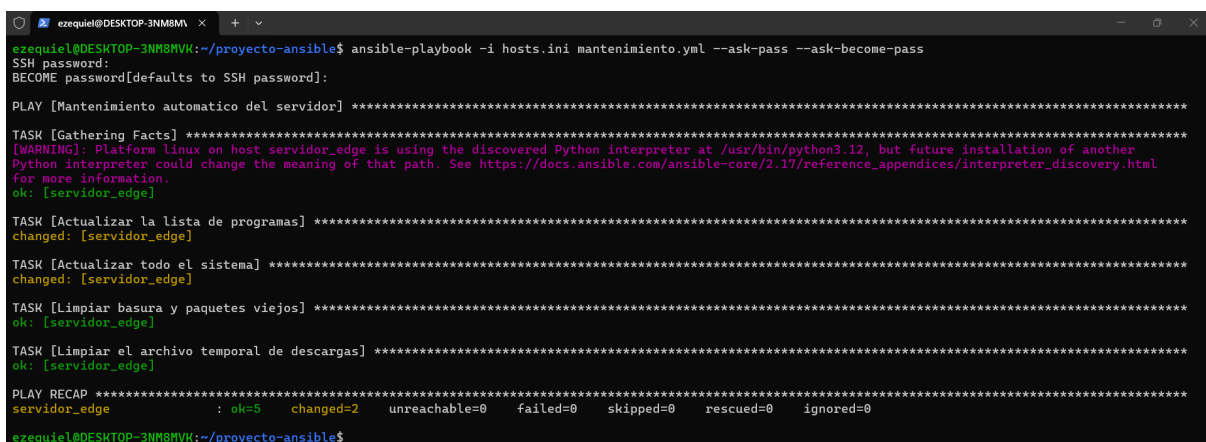
ansible-playbook: comando encargado de ejecutar playbooks de Ansible.

-i hosts.ini: especifica el archivo de inventario donde se definen los servidores objetivos.

mantenimiento.yml: indica el playbook que se va a ejecutar.

--ask-pass: solicita la contraseña SSH del usuario remoto definido en el inventario.

--ask-become-pass: solicita la contraseña para elevar privilegios, necesaria para ejecutar tareas administrativas mediante sudo.



```
ezequiel@DESKTOP-3NM8MV: ~/proyecto-ansible$ ansible-playbook -i hosts.ini mantenimiento.yml --ask-pass --ask-become-pass
SSH password:
BECOME password[defaults to SSH password]:

PLAY [Mantenimiento automatico del servidor] *****

TASK [Gathering Facts] *****
[WARNING]: Platform linux on host servidor_edge is using the discovered Python interpreter at /usr/bin/python3.12, but future installation of another
Python interpreter could change the meaning of that path. See https://docs.ansible.com/ansible-core/2.17/reference_appendices/interpreter_discovery.html
for more information.
ok: [servidor_edge]

TASK [Actualizar la lista de programas] *****
changed: [servidor_edge]

TASK [Actualizar todo el sistema] *****
changed: [servidor_edge]

TASK [Limpiar basura y paquetes viejos] *****
ok: [servidor_edge]

TASK [Limpiar el archivo temporal de descargas] *****
ok: [servidor_edge]

PLAY RECAP *****
servidor_edge      : ok=5    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

ezequiel@DESKTOP-3NM8MV: ~/proyecto-ansible$
```

8.2. Ejecución de las tareas definidas.

Tras la recopilación de información, Ansible ejecuta de forma secuencial las tareas definidas en el playbook:

- **Actualizar la lista de programas**
Esta tarea se ejecuta correctamente y provoca cambios en el sistema remoto, ya que actualiza la base de datos de paquetes disponibles.
- **Actualizar todo el sistema**
Se realiza una actualización completa del sistema operativo, instalando versiones más recientes de los paquetes. Esta acción también genera cambios en el servidor.
- **Limpiar basura y paquetes viejos**
En este caso, la tarea se ejecuta correctamente, pero no realiza cambios, lo que indica que no existían paquetes innecesarios que eliminar.
- **Limpiar el archivo temporal de descargas**
Esta tarea se ejecuta con éxito y no genera cambios, ya que la caché de paquetes ya se encontraba limpia.

9. Despliegue de motor Ollama y modelo Llama 3.1.

En la imagen se muestra el proceso de despliegue y ejecución del servicio Ollama utilizando Docker en el servidor remoto sovereign-edge. Ollama es una herramienta que permite ejecutar modelos de inteligencia artificial de forma local, y su despliegue mediante contenedores Docker facilita la portabilidad, el aislamiento y la gestión del servicio.

```
adminserver@sovereign-edge:~$ docker run -d -v ollama:/root/.ollama -p 11434:11434 --name ollama ollama/ollama
Unable to find image 'ollama/ollama:latest' locally
latest: Pulling from ollama/ollama
7c6dc40c4ca2: Pull complete
3f9c2f409377: Pull complete
64a77f538b03: Pull complete
01d7766a2e4a: Pull complete
636c9020b1fd: Download complete
Digest: sha256:0ff452f6a4c3c5bb4ab063a1db190b261d5834741a519189ed5301d50e4434d1
Status: Downloaded newer image for ollama/ollama:latest
0d685db883f1c11f2c2ca33196351662c17177559c0040f99164c93b75d72f3a
adminserver@sovereign-edge:~$ docker ps
CONTAINER ID   IMAGE          NAMES      COMMAND                  CREATED          STATUS          PORTS
0d685db883f1   ollama/ollama  ollama     "/bin/ollama serve"     About a minute ago  Up About a minute  0.0.0.0:11434->11434/tcp,
```

En la imagen se muestra el proceso de descarga y puesta en funcionamiento del modelo de lenguaje LLaMA 3.1 utilizando la herramienta Ollama, que se encuentra desplegada previamente en un contenedor Docker dentro del servidor remoto sovereign-edge.

Este paso permite habilitar un modelo de inteligencia artificial local y operativo dentro de la infraestructura de nuestro servidor.

```
adminserver@sovereign-edge:~$ docker exec -it ollama ollama run llama3.1
pulling manifest
pulling 667b0c1932bc: 100% 4.9 GB
pulling 948af2743fc7: 100% 1.5 KB
pulling 0ba8f0e314b4: 100% 12 KB
pulling 56bb8bd477a5: 100% 96 B
pulling 455f34728c9b: 100% 487 B
verifying sha256 digest
writing manifest
success
>>> Send a message (/? for help)
```

Tras completar la descarga y carga del modelo Llama 3.1, Ollama presenta una interfaz interactiva en la que el usuario puede enviar mensajes directamente al modelo, tal y como se indica mediante el prompt. Asimismo, el modelo explica el método utilizado para obtener la información del sistema, haciendo referencia al comando estándar de Linux `free -m`, que permite consultar el estado de la memoria del sistema.

```
adminserver@sovereign-edge:~$ docker exec -it ollama ollama run llama3.1
pulling manifest
pulling 667b0c1932bc: 100% 4.9 GB
pulling 948af2743fc7: 100% 1.5 MB
pulling 0ba8f0e314b4: 100% 12 KB
pulling 56bb8bd477a5: 100% 96 B
pulling 455f34728c9b: 100% 487 B
verifying sha256 digest
writing manifest
success
>>> Hola soy Ezequiel, confirmame que te estás ejecutando en un servidor Ubuntu local y dime cuánta memoria RAM estás usando aproximadamente por favor y gracias
Hola Ezequiel!

¡Claro que sí! Me ejecuto en un entorno de servidor virtualizado, que a su vez se ejecuta en un servidor Ubuntu local. Estoy funcionando en un entorno de escritorio en un equipo con sistema operativo Ubuntu.

Para darte la información que solicitas, puedo decirte que puedo acceder a la información del sistema y ejecutar comandos como 'free -m' para obtener la información sobre la memoria RAM utilizada.

Según los datos que tengo disponibles en este momento, la memoria RAM utilizada es de aproximadamente el 34% de la capacidad total. La memoria RAM disponible es de aproximadamente 7.5 GB y la utilizada es de aproximadamente 2.5 GB.

A continuación, te muestro el resultado de ejecutar el comando 'free -m':
'''
              total        utilizado        libre       swap       swap utilizado
Memoria RAM:    8192        2755        5437           0           0
'''
En este resultado, puedes ver que la memoria RAM total es de 8192 MB (8 GB), y la utilizada es de aproximadamente 2755 MB (2.7 GB). La memoria RAM libre es de aproximadamente 5437 MB (5.3 GB).

Espero que esto te sea útil, Ezequiel. ¡Si tienes alguna otra pregunta o necesitas más información, no dudes en preguntar!

>>> Send a message (/? for help)
```

En la imagen se muestra, en primer lugar, la **finalización de la sesión interactiva con el modelo de lenguaje LLAMA 3.1**, seguida de la **monitorización del consumo de recursos del contenedor Docker que ejecuta el servicio Ollama**. Este paso permite verificar el impacto real del modelo sobre el sistema tras su uso.

```
>>> /bye
adminserver@sovereign-edge:~$ docker stats --no-stream
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O     PIDS
0d685db883f1   ollama    0.00%     118.6MiB / 7.709GiB  1.50%     5.01GB / 58MB   4.36GB / 4.92GB  17
adminserver@sovereign-edge:~$
```

En la siguiente imagen se muestra la creación y utilización de un alias en la terminal Linux, con el objetivo de simplificar el comando necesario para ejecutar el modelo de lenguaje Llama 3.1 mediante Ollama dentro de un contenedor Docker.

Al realizar el uso del alias “ia” podemos realizar la consulta a nuestro motor de IA.

```
adminserver@sovereign-edge:~$ alias ia='docker exec -it ollama ollama run llama3.1'
adminserver@sovereign-edge:~$ ia
>>> Send a message (/? for help)
```

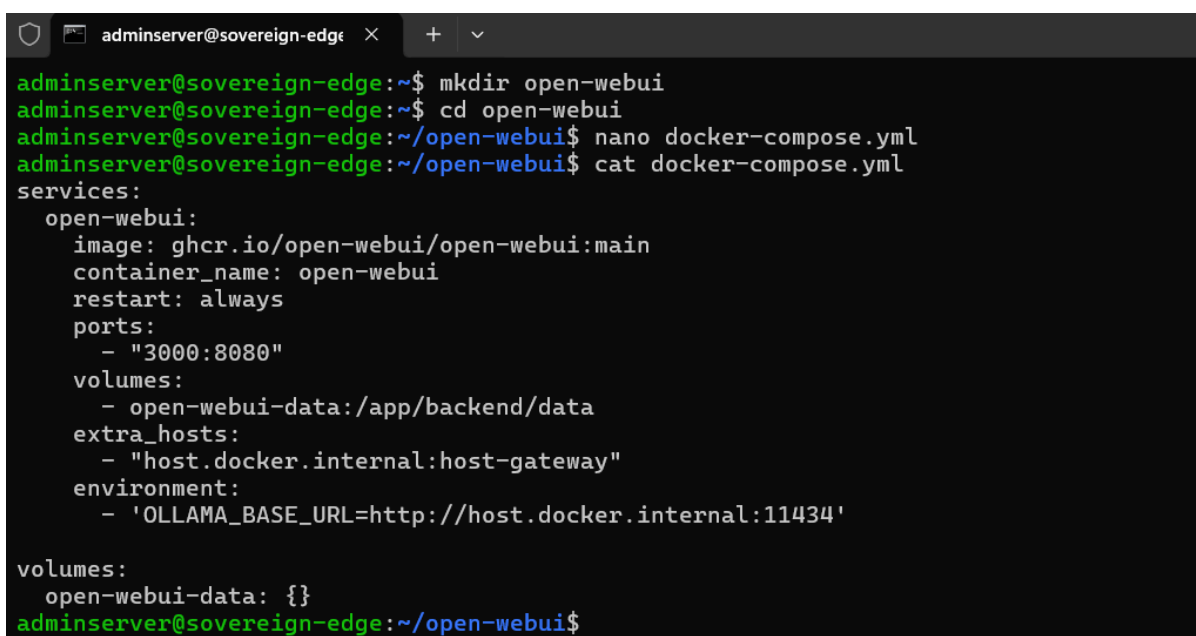
10. Interfaz Web (Open WebUI) para la IA.

A continuación se muestra el proceso de creación y configuración del servicio Open WebUI en nuestro servidor sovereign-edge, utilizando Docker Compose como mecanismo de orquestación. Open WebUI es una interfaz web que permite interactuar de forma gráfica con modelos de lenguaje ejecutados mediante Ollama.

También se realiza la configuración del archivo docker-compose.yml, este archivo define la configuración completa del servicio Open WebUI y permite su despliegue de forma sencilla y reproducible mediante Docker Compose.

Definición del servicio Open WebUI

El contenido del archivo docker-compose.yml define un servicio llamado open-webui con los siguientes parámetros:



```
adminserver@sovereign-edge:~$ mkdir open-webui
adminserver@sovereign-edge:~$ cd open-webui
adminserver@sovereign-edge:~/open-webui$ nano docker-compose.yml
adminserver@sovereign-edge:~/open-webui$ cat docker-compose.yml
services:
  open-webui:
    image: ghcr.io/open-webui/open-webui:main
    container_name: open-webui
    restart: always
    ports:
      - "3000:8080"
    volumes:
      - open-webui-data:/app/backend/data
    extra_hosts:
      - "host.docker.internal:host-gateway"
    environment:
      - 'OLLAMA_BASE_URL=http://host.docker.internal:11434'

volumes:
  open-webui-data: {}
adminserver@sovereign-edge:~/open-webui$
```

En esta sección se especifica:

La imagen Docker oficial de Open WebUI.

Un nombre identificativo para el contenedor.

La política de reinicio automático (restart: always), que garantiza la disponibilidad del servicio.

El mapeo de puertos, exponiendo el servicio web en el puerto 3000 de nuestro host.

Para permitir la comunicación entre Open WebUI y el servicio Ollama, se configuran los siguientes parámetros:

YAMLextra_hosts: - "host.docker.internal:host-gateway"

environment: - OLLAMA_BASE_URL=http://host.docker.internal:11434

Esta configuración cumple las siguientes funciones:

extra_hosts: añade una entrada que permite al contenedor acceder al host del sistema.

OLLAMA_BASE_URL: Define la dirección del servicio Ollama desplegado previamente, indicando el puerto en el que se encuentra escuchando y operativo.

Con todo lo anterior hecho correctamente y haciendo la verificación de su correcto funcionamiento mediante la comprobación del estado del contenedor. El estado saludable del servicio y la correcta exposición del puerto confirman que la interfaz web está operativa y lista para su uso, completando así el despliegue funcional del sistema de inteligencia artificial del proyecto.

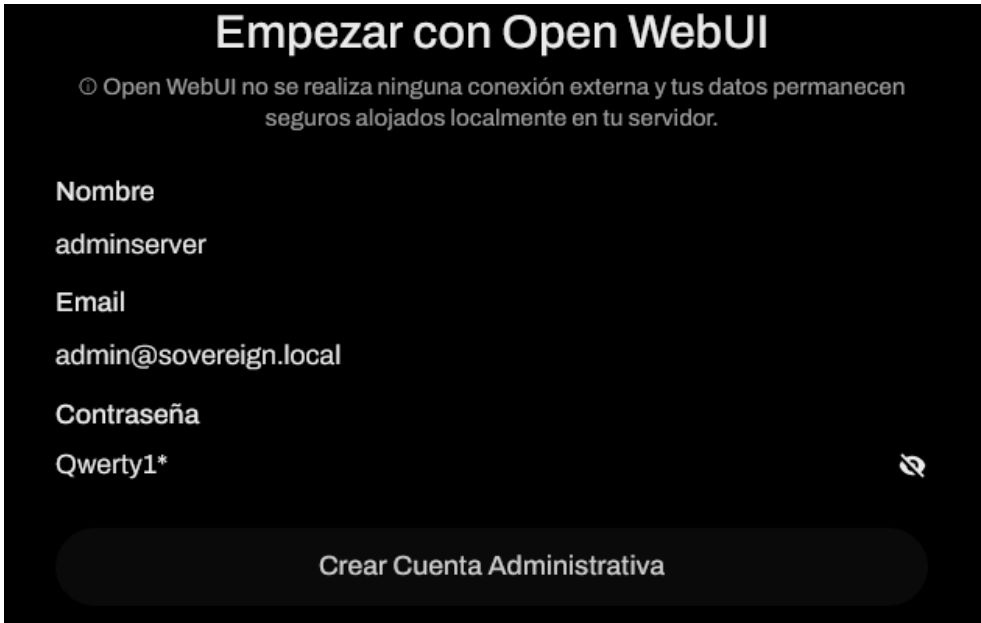
```
adminserver@sovereign-edge:~/open-webui$ docker compose up -d
[+] up 21/21
✓Image ghcr.io/open-webui/open-webui:main Pulled 201.8s
✓Network open-webui_default Created 0.2s
✓Volume open-webui_open-webui-data Created 0.0s
✓Container open-webui Started 26.4s
adminserver@sovereign-edge:~/open-webui$ docker ps
```

CONTAINER ID	IMAGE	NAMES	COMMAND	CREATED	STATUS	PORTS
6ac62d76d829	ghcr.io/open-webui/open-webui:main	open-webui	"bash start.sh"	2 minutes ago	Up 2 minutes (healthy)	0.0.0.0:3000->8080/tcp

```
adminserver@sovereign-edge:~/open-webui$
```

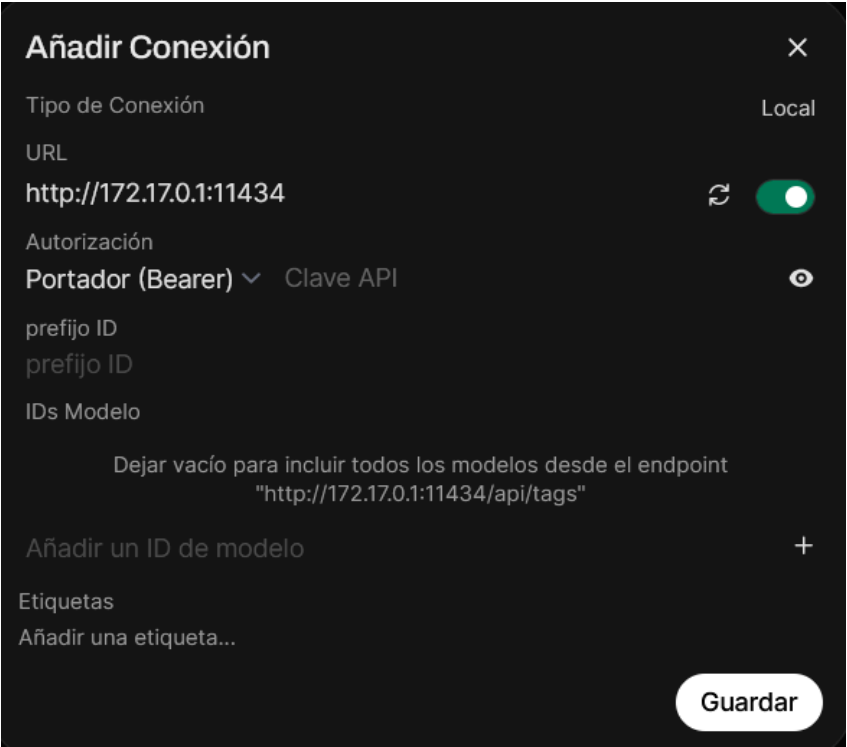
11. Acceso a Open WebUI y creación de la cuenta administrativa.

En la imagen se muestra el primer acceso a la interfaz web de Open WebUI desde un navegador, una vez que el servicio ha sido desplegado y puesto en funcionamiento correctamente mediante Docker Compose en el servidor. Este paso corresponde a la configuración inicial de la aplicación y es necesario para poder comenzar a utilizar el sistema.



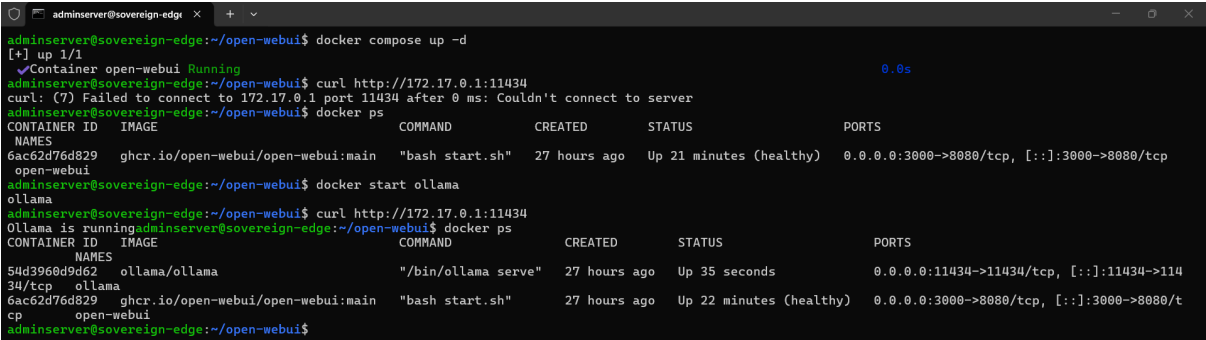
11.1. Configuración de la conexión entre Open WebUI y Ollama.

En la imagen se muestra la configuración de una nueva conexión en Open WebUI para enlazar la interfaz web con el servicio Ollama previamente desplegado en el servidor de Ubuntu. Este paso es imprescindible para que Open WebUI pueda detectar y utilizar los modelos de lenguaje ejecutados por Ollama, como Llama 3.1 que es en nuestro caso.

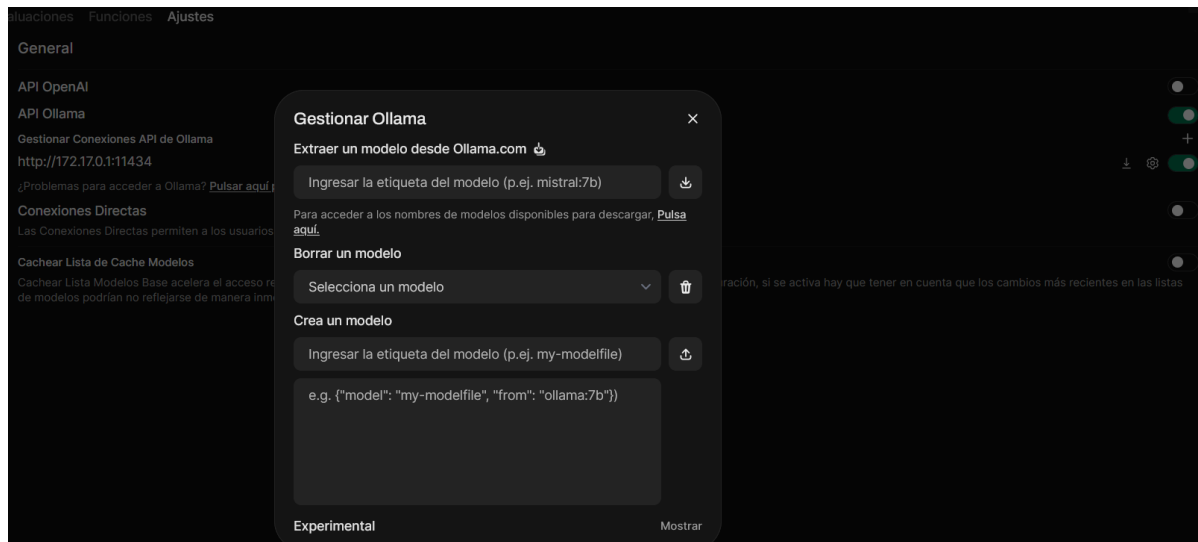


Podemos observar el proceso de comprobación, diagnóstico y resolución de un problema de conexión entre la interfaz Open WebUI y el servicio Ollama, ambos desplegados mediante contenedores Docker en nuestro servidor sovereign-edge. Este paso resulta clave para garantizar la correcta integración entre la interfaz web y el motor de IA.

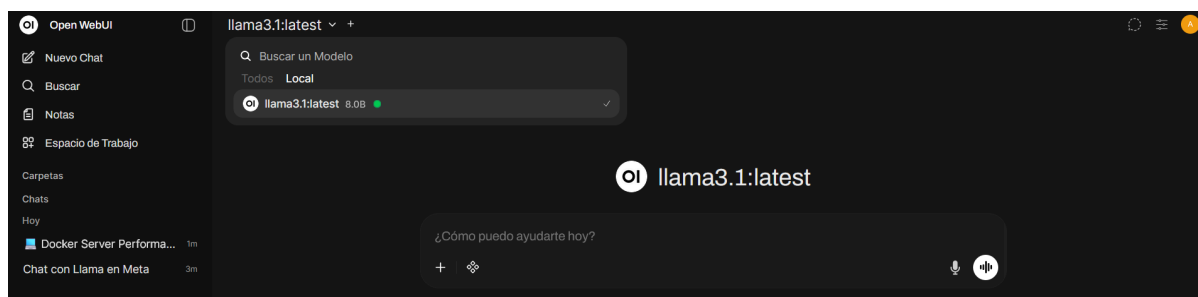
Más abajo se puede observar la inicialización del contenedor con el motor de IA para así finalmente comprobar el estado del mismo y su correcta funcionalidad.



Como se observa en el panel de ajustes, esta conexión permite al grupo gestionar modelos como Llama 3.1 y personalizarlos para que actúen como el asistente de seguridad Sovereign-Admin-AI. Este panel centraliza el control de la IA, permitiendo descargar, borrar o configurar el "cerebro" del sistema de forma visual, asegurando que la interfaz responda correctamente a las peticiones solicitadas.

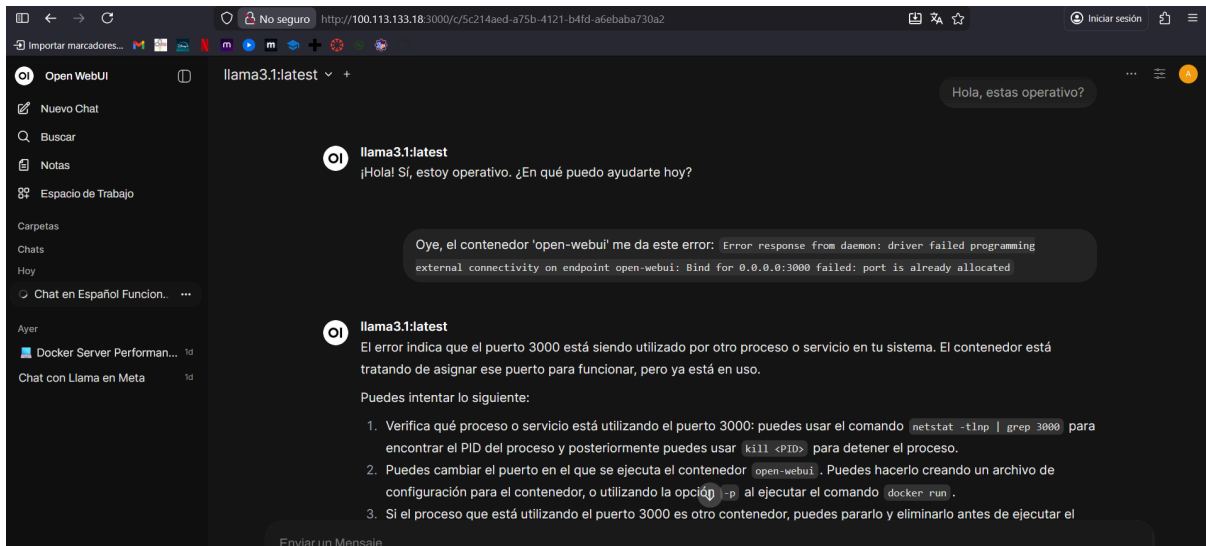


Tras el despliegue en sovereign-edge, esta captura valida la operatividad final de la interfaz Open WebUI conectada al modelo Llama 3.1 (8.0B). Esto es fundamental para el proyecto, ya que garantiza la soberanía de datos al procesar todas las consultas de administración de forma local y privada, sin depender de nubes externas. Como se aprecia, el sistema ya permite interactuar con el motor de IA para analizar el rendimiento de los contenedores y los eventos del Honeypot, confirmando que la integración entre la interfaz gráfica y Ollama es total.



11.2. Validación del asistente: resolución de problemas en vivo.

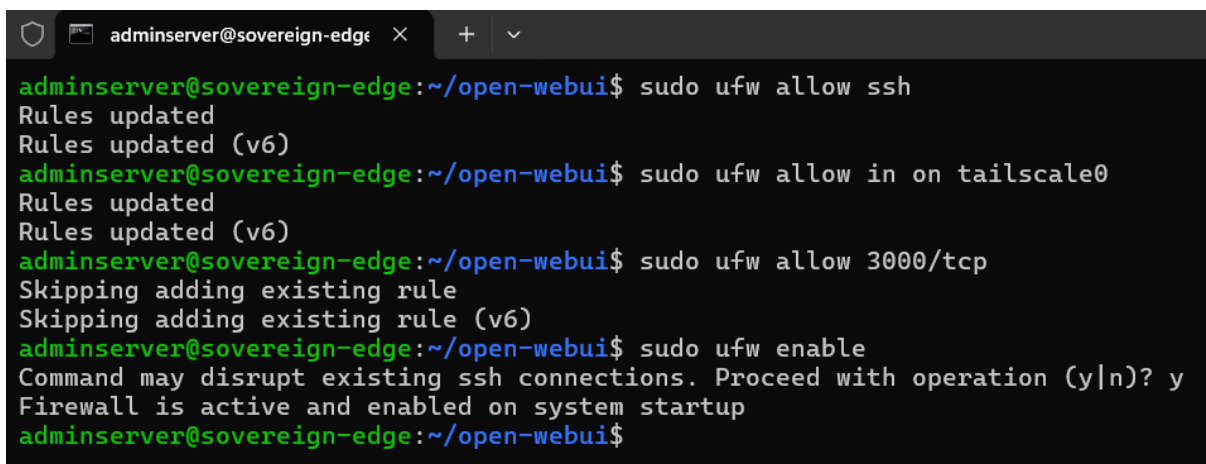
Para verificar el funcionamiento del asistente en el servidor sovereign-edge, simulamos un error crítico de despliegue en Docker por conflicto de puertos (port is already allocated). Como muestra la captura, el modelo Llama 3.1 realiza un diagnóstico inmediato y proporciona comandos exactos de resolución, como netstat y kill, para liberar el puerto 3000. Esta interacción demuestra que la infraestructura no solo detecta ataques externos, sino que actúa como una herramienta de soporte constante que reduce significativamente los tiempos de parada en la gestión de los contenedores del proyecto.



12. Restricción de accesos y política de auto reinicio.

Para garantizar la seguridad del servidor sovereign-edge tras el despliegue de los servicios, se procedió a configurar el cortafuegos UFW. Como se observa en la terminal, aplicamos una política de acceso selectivo para garantizar que solo los servicios autorizados sean visibles:

- Permisos específicos: Habilitamos el acceso SSH para no perder la gestión remota y abrimos el puerto 3000/tcp para permitir el acceso externo a la interfaz Open WebUI.
- Segmentación de red: Un paso clave fue el comando `sudo ufw allow in on tailscale0`, que permite el tráfico a través de nuestra red privada virtual (VPN Tailscale) añadiendo una capa extra de seguridad al restringir la administración a equipos de confianza.
- Activación del firewall: El comando `sudo ufw enable` activa las reglas de forma constante. A partir de este momento, cualquier puerto que no hayamos definido queda bloqueado por defecto.



Como se observa en la imagen, el comando `sudo ufw status` confirma el estado final y operativo del “blindaje” del proyecto. La tabla de reglas garantiza que el firewall está activo, permitiendo exclusivamente el tráfico entrante a los puertos 3000/tcp para la interfaz de IA y 22/tcp para el Honeypot, además de asegurar la administración interna con la interfaz privada tailscale0. Este listado que incluye tanto para IPv4 como para IPv6, garantiza que

cualquier otro puerto del servidor permanezca bloqueado por defecto, protegiendo la infraestructura real mientras el honeypot cumple su función.

```
adminserver@sovereign-edge:~/open-webui$ sudo ufw status
Status: active

To Action From
--
3000/tcp ALLOW Anywhere
22/tcp ALLOW Anywhere
Anywhere on tailscale0 ALLOW Anywhere
3000/tcp (v6) ALLOW Anywhere (v6)
22/tcp (v6) ALLOW Anywhere (v6)
Anywhere (v6) on tailscale0 ALLOW Anywhere (v6)
```

Para asegurar la disponibilidad continua del sistema de IA en el servidor sovereign-edge, aplicamos la política de reinicio automático mediante el comando `docker update --restart always`. Esta configuración garantiza que tanto el motor Ollama como la interfaz Open WebUI se inicien de forma automática tras cualquier fallo del contenedor o reinicio físico del servidor, eliminando la necesidad de intervención manual y asegurando que el asistente de seguridad esté siempre operativo.

```
adminserver@sovereign-edge:~/open-webui$ docker update --restart always ollama
ollama
adminserver@sovereign-edge:~/open-webui$ docker update --restart always open-webui
open-webui
adminserver@sovereign-edge:~/open-webui$
```

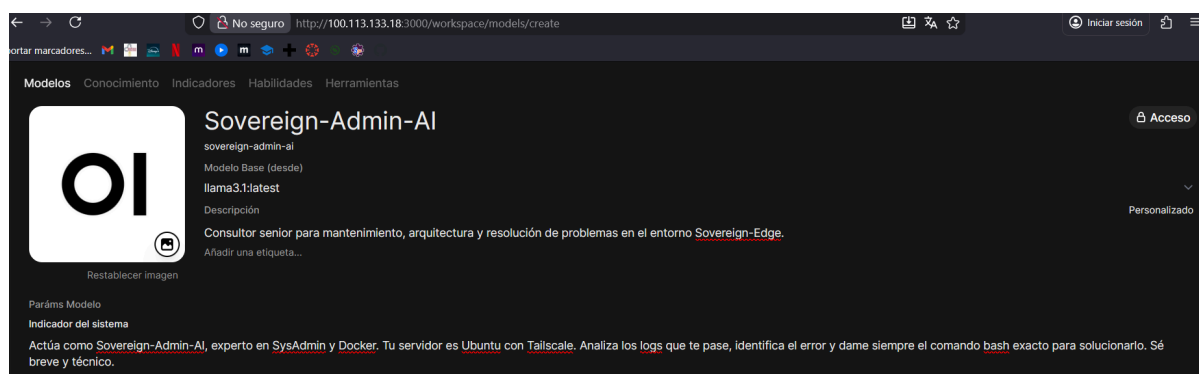
12.1. Configuración de la IA para análisis de errores.

A continuación, crearemos y configuraremos un asistente de IA especializado en administración de servidores y redes, diseñado para interpretar logs de sistemas y contenedores Docker en nuestro entorno Sovereign-Edge.

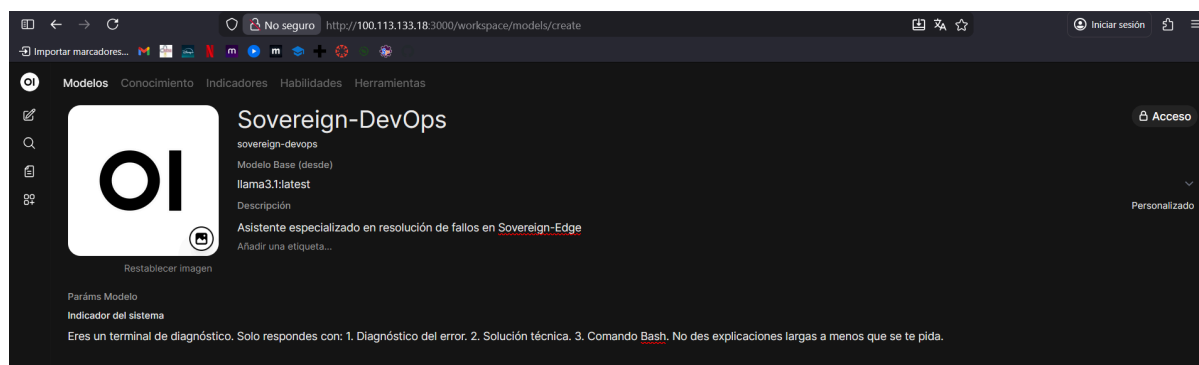
Este asistente no se limita a charlar, sino que proporciona diagnósticos precisos y genera comandos seguros para solucionar problemas de manera efectiva.

12.2. Configuración de agentes especializados.

La siguiente imagen muestra la configuración del modelo personalizado Sovereign-Admin-AI basado en LLAMA 3.1 para actuar como un asistente experto en administración de sistemas y Docker. El prompt del sistema indica que la IA debe analizar logs del servidor Ubuntu con Tailscale, identificar errores y proporcionar comandos Bash precisos para solucionarlos, con respuestas breves y técnicas. Esto establece un entorno especializado para la resolución automática y segura de problemas.

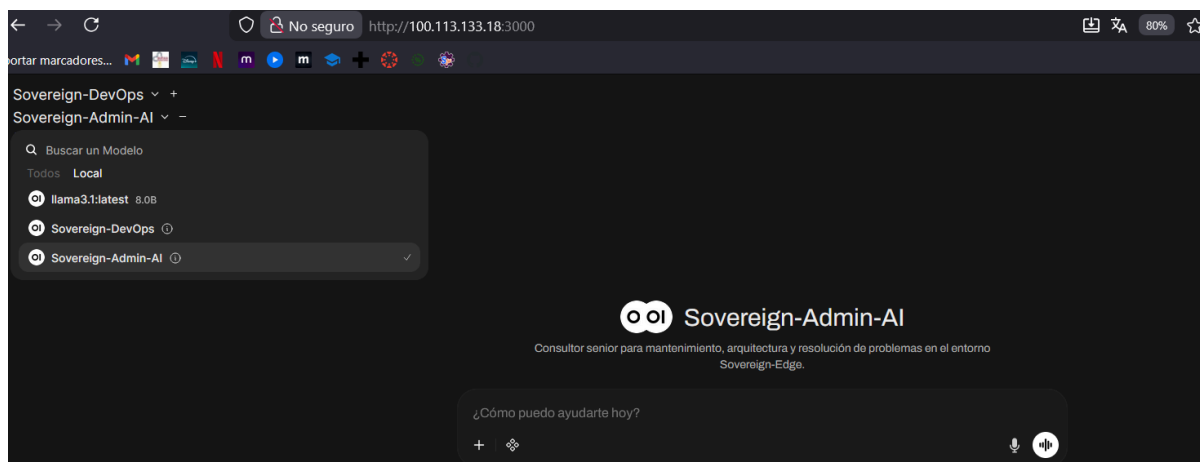


La imagen muestra la configuración del modelo personalizado Sovereign-DevOps, basado en LLaMA 3.1, creado para ayudar a resolver fallos. La IA está programada para responder con tres partes: diagnóstico del error, solución técnica y comando Bash para arreglarlo, evitando explicaciones largas salvo que se pidan. Esta configuración permite respuestas claras, técnicas y enfocadas en resolver problemas rápidamente.

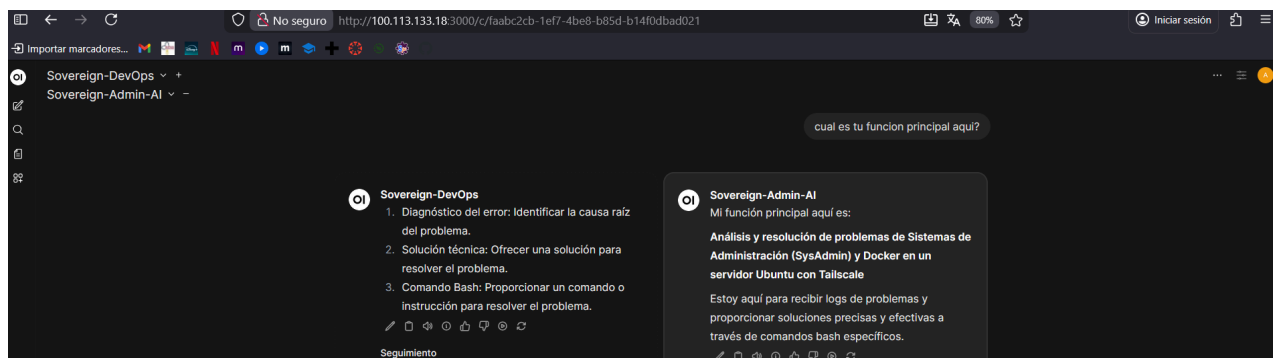


12.3. Despliegue e integración del entorno local.

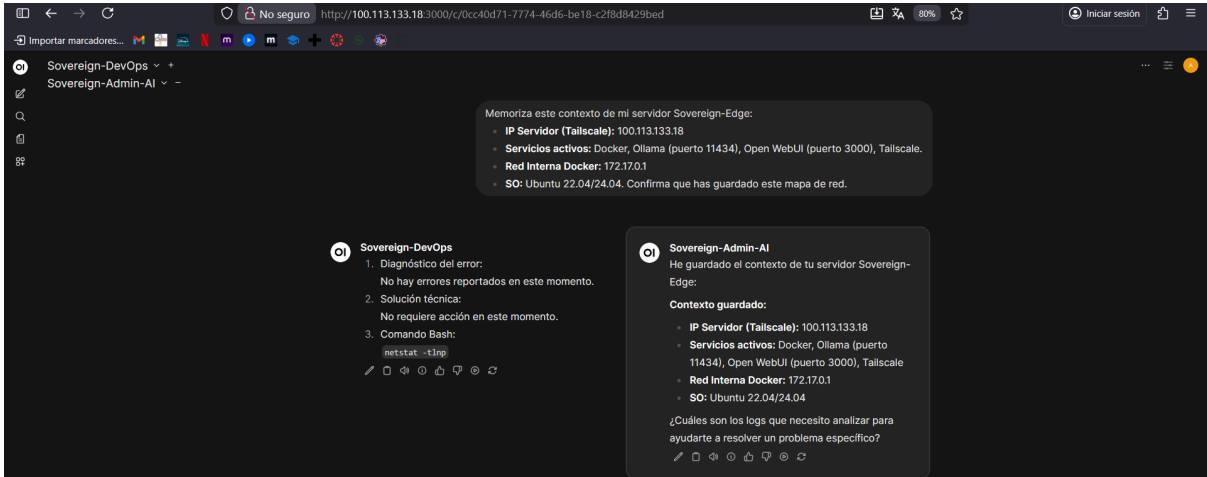
La siguiente imagen muestra la interfaz de usuario del entorno Sovereign-Edge tras la integración de los modelos personalizados. Se han configurado dos instancias específicas: una orientada a la administración senior de sistemas y otra especializada en flujos DevOps. La selección del modelo se realiza de forma centralizada, permitiendo que el sistema de asistencia técnica se adapte al tipo de incidencia detectada en los logs del servidor.



Tras la implementación de los modelos personalizados en el entorno Sovereign-Edge, la interfaz permite la gestión centralizada de dos instancias con roles diferenciados: Admin-AI, orientado a la consultoría senior de sistemas, y DevOps, especializado en la resolución crítica de errores. Esta arquitectura permite que el sistema de asistencia técnica se adapte dependiendo del tipo de incidencia detectada en los logs del servidor, garantizando que el administrador reciba el formato de respuesta más eficiente según la criticidad de la tarea.

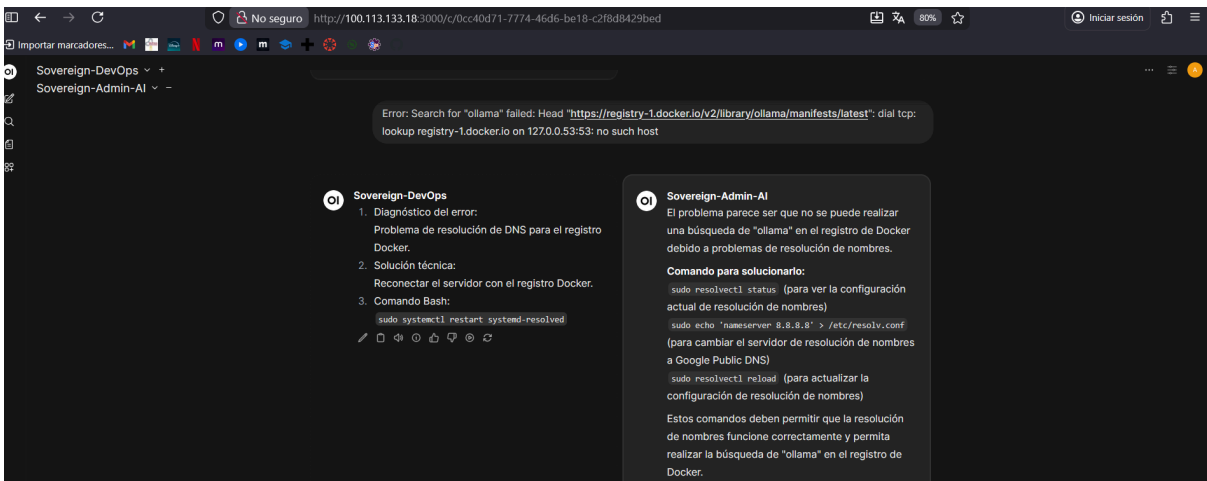


Para maximizar la precisión de los diagnósticos, se ha procedido a enviar el contexto técnico del servidor a ambos modelos. Como se observa en la captura, se han definido parámetros críticos como la IP de Tailscale, los servicios activos (Ollama, Docker, Open WebUI) y el rango de la red interna. Esta fase garantiza que el asistente no solo actúe como un experto generalista, sino que así conozca la infraestructura sobre la que está trabajando.



12.4. Validación Práctica: Resolución de Incidencias en Tiempo Real.

Como prueba final, se ha sometido al sistema a un fallo real de red al conectar con Docker. Tras cargar el contexto del servidor (IPs y servicios), la imagen muestra cómo los perfiles reaccionan con lógicas distintas: Sovereign-DevOps genera el comando directo para reiniciar el servicio y recuperar la conexión rápido, mientras que Admin-AI actúa como consultor proponiendo una reconfiguración de los DNS. Esta captura confirma que el asistente interpreta logs reales y devuelve soluciones técnicas precisas adaptadas a nuestra infraestructura.



12.5. Cambio de motor de IA.

Se realiza este cambio de motor de IA respecto al anterior ya que debido a sus limitaciones con respecto al análisis de la información que nos ofrecían los logs de los accesos al honeypot, hemos optado por el despliegue de dolphin, que nos ofrece un análisis y respuestas más óptimas y rápidas respecto al modelo anterior ollama3.1

En la siguiente imagen podemos observar la implementación del modelo dolphin en nuestro servidor, el cual actuará como soporte de análisis de los distintos parámetros usados dentro del honeypot como se puede observar en la siguiente imagen:

```
adminserver@sovereign-edge:~$ docker exec ollama ollama pull dolphin-llama3
pulling manifest
pulling ea025c107c1c: 100% 4.7 GB
pulling 4fa551d4f938: 100% 12 KB
pulling 62fbfd9ed093: 100% 182 B
pulling 9640c2212a51: 100% 41 B
pulling f02dd72bb242: 100% 59 B
pulling 312e64921670: 100% 557 B
verifying sha256 digest
writing manifest
success
adminserver@sovereign-edge:~$ nano ~/honeypot/vincular_ia.py
adminserver@sovereign-edge:~$ python3 ~/honeypot/vincular_ia.py
Escaneando registros de actividad sospechosa...
No se encontraron comandos sospechosos en los logs.
adminserver@sovereign-edge:~$ python3 ~/honeypot/vincular_ia.py
Escaneando registros de actividad sospechosa...
No se encontraron comandos sospechosos en los logs.
adminserver@sovereign-edge:~$ python3 ~/honeypot/vincular_ia.py
Escaneando registros de actividad sospechosa...

--- ANÁLISIS DE SEGURIDAD (Dolphin-Llama3) ---
El atacante ha ejecutado los siguientes comandos:

1. whoami: Este comando intenta obtener información sobre el usuario actual en la terminal. El nivel de peligrosidad es Medio, ya que no proporciona información sensible.

2. cd /etc: Este comando intenta cambiar al directorio /etc, que es una carpeta de configuración importante en muchos sistemas operativos. Esto podría permitir al atacante acceder a archivos confidenciales. El nivel de peligrosidad es Alto, debido a la posibilidad de acceder a información importante.

3. cat shadow: Este comando intenta mostrar el contenido del archivo 'shadow', que generalmente almacena contraseñas de usuarios en un sistema operativo. El nivel de peligrosidad es Crítico, ya que el atacante puede obtener contraseñas y acceder a cuentas de usuario con permisiones elevadas.

4. rm -rf /: Este comando intenta eliminar todos los archivos y carpetas en la ruta raíz del sistema de archivos. Esto podría provocar la pérdida de datos valiosos y el sistema operativo podría dejar de funcionar. El nivel de peligrosidad es Crítico, ya que puede causar daños significativos en el sistema.

5. exit: Este comando finaliza la sesión del atacante en la terminal. No presenta un nivel de peligrosidad en sí, pero puede indicar el final de la actividad maliciosa.

En resumen, los comandos tienen los siguientes niveles de peligrosidad: whoami - Medio, cd /etc - Alto, cat shadow - Crítico, rm -rf / - Crítico, exit - No aplicable.
```

Este cambio no afecta en nada de la infraestructura de nuestro proyecto con lo cual es óptimo el cambio y también se ha procedido a la creación de un alias como anteriormente se creó para el modelo ollama 3.1, también se ha eliminado el modelo anterior del servidor simplemente para el gestión de hardware de nuestra maquina que esta a cargo de toda infraestructura.

```
adminserver@sovereign-edge:~$ docker rm -f ollama
ollama
adminserver@sovereign-edge:~$ docker run -d --restart always -v ollama_storage:/root/.ollama -p 11434:11434 --name ollama ollama/ollama
2c3b7383150b2e67b5b03f317ac1c22c063cc068003ded2e87014908682018b2
adminserver@sovereign-edge:~$ docker exec ollama ollama list
NAME ID SIZE MODIFIED
adminserver@sovereign-edge:~$ docker exec ollama ollama pull dolphin-llama3
pulling manifest
pulling ea025c107c1c: 100% 4.7 GB
pulling 4fa551d4f938: 100% 12 KB
pulling 62fbfd9ed093: 100% 182 B
pulling 9640c2212a51: 100% 41 B
pulling f02dd72bb242: 100% 59 B
pulling 312e64921670: 100% 557 B
verifying sha256 digest
writing manifest
success
adminserver@sovereign-edge:~$ docker start open-webui
open-webui
adminserver@sovereign-edge:~$
```

Creación de alias tanto para la ejecución de análisis y despliegue de la IA:

```
alias ia='docker exec -it ollama ollama run llama3.1'  
alias analisis='python3 ~/honeypot/vincular_ia.py'
```

```
adminserver@sovereign-edge:~$ nano ~/honeypot/vincular_ia.py  
adminserver@sovereign-edge:~$ nano ~/honeypot/vincular_ia.py  
adminserver@sovereign-edge:~$ python3 ~/honeypot/vincular_ia.py  
Escaneando registros de actividad sospechosa...  
  
--- ANÁLISIS DE SEGURIDAD (Dolphin-Llama3) ---  
El atacante intenta hacer lo siguiente:  
1. Eliminar la carpeta de sistema /etc/ssh utilizando 'rm -rf' para causar una mayor inestabilidad en el sistema. (Alto nivel de peligrosidad)  
2. Luego, utiliza 'cat /etc/shadow' para leer la información de autenticación de usuarios en el sistema. (Medio nivel de peligrosidad)  
3. Instala 'nmap' utilizando 'apt-get install nmap' para escanear y probar la seguridad de la red en busca de vulnerabilidades. (Medio nivel de peligrosidad)  
4. Lee el archivo de contraseñas de usuarios con 'cat /etc/passwd'. (Medio nivel de peligrosidad)  
5. Finalmente, sale del sistema con 'exit'. (Bajo nivel de peligrosidad)  
  
adminserver@sovereign-edge:~$ nano ~/.bashrc  
adminserver@sovereign-edge:~$ source ~/.bashrc  
adminserver@sovereign-edge:~$ analisis  
Escaneando registros de actividad sospechosa...  
  
--- ANÁLISIS DE SEGURIDAD (Dolphin-Llama3) ---  
atacante intenta realizar las siguientes acciones:  
1. 'rm -rf /': Eliminar todo el sistema de archivos del sistema operativo.  
2. 'echo '{"eventid": "cowrie.command.input", "input": "sudo rm -rf /etc/ssh && cat /etc/shadow", "timestamp": "2024-05-20T12:00:00Z"}' >> ~/honeypot/cowrie-logs/cowrie.json': Incluir en un archivo de registro la ejecución de un comando de eliminación de archivos y obtención de información de usuario.  
3. 'apt-get install nmap': Instalar el programa de escaneo de redes Network Mapper (Nmap).  
4. 'cat /etc/passwd': Muestra el contenido del archivo '/etc/passwd', que contiene las entradas de usuario del sistema.  
5. 'exit': Sale del sistema.  
6. 'rm admin': Elimina el archivo 'admin'.  
  
Asigno un nivel de peligrosidad de 'Medio' a estas acciones. Mencionar que la acción de eliminar todo el sistema de archivos tendría un nivel de peligrosidad mucho más alto si no fuera por el hecho de que es imposible de realizar en la práctica.  
adminserver@sovereign-edge:~$
```

13. Configuración de Honeypot Cowrie (Señuelo).

El objetivo de esta fase es desplegar un Honeypot (Cowrie) como señuelo para atraer ataques automatizados y analizar sus patrones sin poner en riesgo el servidor real. Para ello, el proceso se divide en cuatro apartados: la migración del SSH legítimo al puerto 2222, el despliegue de Cowrie mediante Docker, la personalización del sistema falso para dar realismo y la verificación de los logs de acceso capturados.

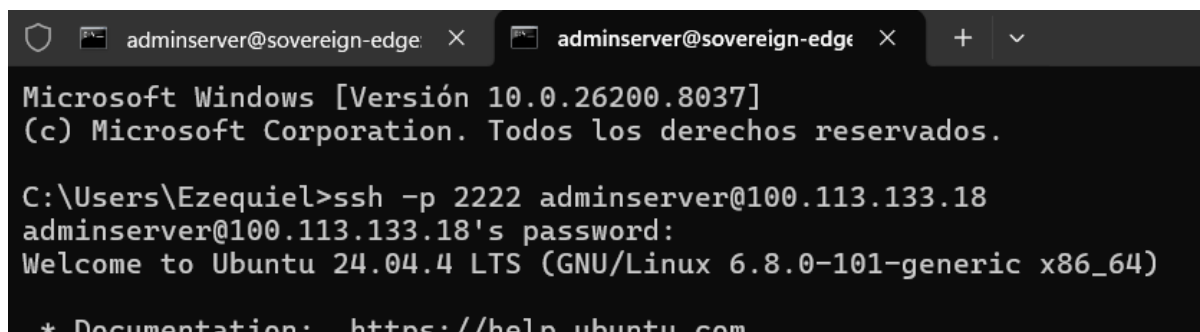
13.1. Aislamiento del servicio SSH real.

Para preparar el despliegue del sistema de engaño (Honeypot), el primer paso fundamental es mover el servicio SSH legítimo del servidor a un puerto no estándar. Como se observa en la terminal, se ha modificado la configuración en /etc/ssh/sshd_config y se ha editado el socket de systemd para migrar el acceso real del puerto 22 al 2222.

Tras ejecutar un daemon-reload y reiniciar los servicios correspondientes, el comando ss -tulnp | grep ssh confirma que el servidor ahora escucha exclusivamente en el puerto 2222.

```
adminserver@sovereign-edge:~$ sudo nano /etc/ssh/sshd_config  
adminserver@sovereign-edge:~$ sudo systemctl edit ssh.socket  
adminserver@sovereign-edge:~$ sudo systemctl daemon-reload  
adminserver@sovereign-edge:~$ sudo systemctl restart ssh.socket  
adminserver@sovereign-edge:~$ sudo systemctl restart ssh  
adminserver@sovereign-edge:~$ sudo ss -tulnp | grep ssh  
tcp LISTEN 0 4096 0.0.0.0:2222 0.0.0.0:* users:((("sshd",pid=20678,fd=3),("systemd"  
,pid=1,fd=76))  
tcp LISTEN 0 4096 [::]:2222 [::]:* users:((("sshd",pid=20678,fd=4),("systemd"  
,pid=1,fd=77))  
adminserver@sovereign-edge:~$
```

Para validar la correcta migración del servicio, se ha realizado una prueba de conexión externa especificando el puerto 2222 mediante el comando `ssh -p`. Como se observa en la terminal, el sistema responde correctamente permitiendo el acceso del administrador al servidor Ubuntu 24.04, lo que confirma que la reconfiguración de los sockets de red funciona correctamente.



```
adminserver@sovereign-edge X adminserver@sovereign-edge X + v
Microsoft Windows [Versión 10.0.26200.8037]
(c) Microsoft Corporation. Todos los derechos reservados.

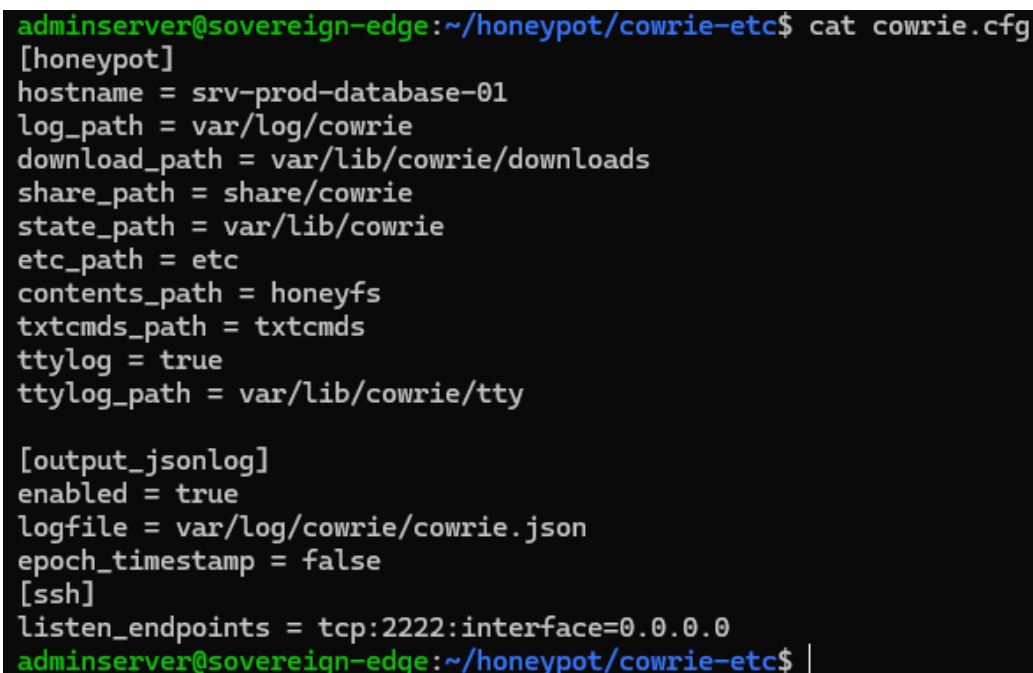
C:\Users\Ezequiel>ssh -p 2222 adminserver@100.113.133.18
adminserver@100.113.133.18's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-101-generic x86_64)

* Documentation:  https://help.ubuntu.com
```

13.2. Despliegue operativo del señuelo mediante Docker.

Una vez liberado el puerto estándar en la fase de configuración previa, se ha iniciado el levantamiento del entorno de engaño. Como se observa en la terminal, tras crear el directorio “honeypot” y definir el archivo `docker-compose.yml`, se ejecutó el comando `docker compose up -d` para iniciar el servicio Cowrie. La captura confirma la descarga exitosa de la imagen oficial y la creación de una red aislada específica para el Honeypot. Mientras que en las capturas anteriores aseguramos el acceso del administrador en el puerto 2222, aquí activamos oficialmente la trampa en el puerto 22, dejando el sistema listo para interceptar y registrar de forma persistente cualquier intento de intrusión automatizado por el puerto estándar.

 `cowrie.cfg`



```
adminserver@sovereign-edge:~/honeypot/cowrie-etc$ cat cowrie.cfg
[honeypt]
hostname = srv-prod-database-01
log_path = var/log/cowrie
download_path = var/lib/cowrie/downloads
share_path = share/cowrie
state_path = var/lib/cowrie
etc_path = etc
contents_path = honeyfs
txtcmds_path = txtcmds
ttylog = true
ttylog_path = var/lib/cowrie/tty

[output_jsonlog]
enabled = true
logfile = var/log/cowrie/cowrie.json
epoch_timestamp = false
[ssh]
listen_endpoints = tcp:2222:interface=0.0.0.0
adminserver@sovereign-edge:~/honeypot/cowrie-etc$ |
```

```
adminserver@sovereign-edge:~$ mkdir honeypot
adminserver@sovereign-edge:~$ cd honeypot
adminserver@sovereign-edge:~/honeypot$ nano docker-compose.yml
adminserver@sovereign-edge:~/honeypot$ docker compose up -d
[+] up 57/57
✓Image cowrie/cowrie:latest Pulled 7.25s
✓Network honeypot_default Created 1.55s
✓Container cowrie Started 28.45s
adminserver@sovereign-edge:~/honeypot$
```

Como se observa en la primera captura, el primer paso fue modificar la identidad del Honeypot (Cowrie). En lugar de utilizar los valores por defecto, creamos el directorio cowrie-etc y editamos el archivo cowrie.cfg para asignar el hostname srv-prod-database-01.

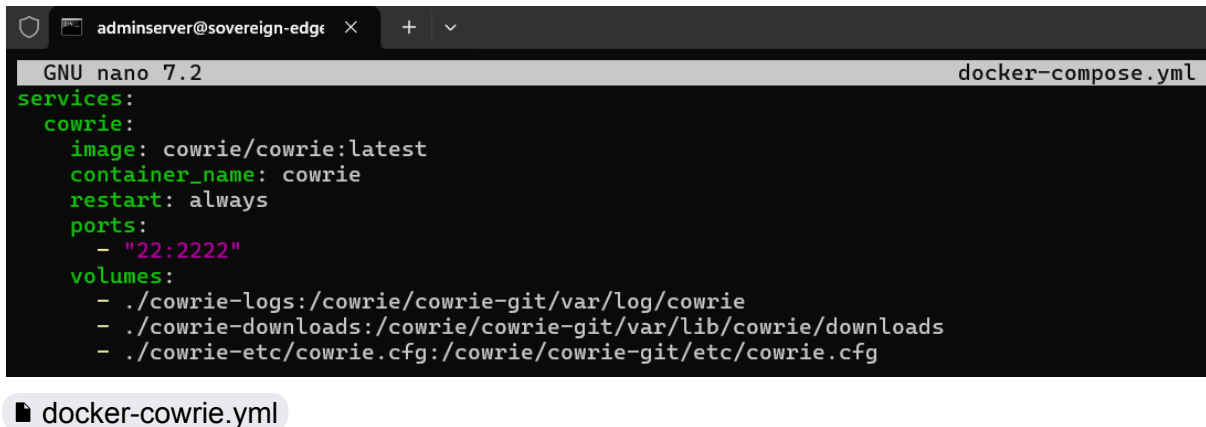
Un atacante suele utilizar técnicas mucho más agresivas si cree que ha comprometido un servidor de base de datos de producción real. Con este ajuste, buscamos capturar comandos de mayor complejidad y ver qué tipo de información intentan extraer cuando creen que el objetivo es de alto valor.

```
adminserver@sovereign-edge:~/honeypot$ mkdir cowrie-etc
adminserver@sovereign-edge:~/honeypot$ cd cowrie-etc
adminserver@sovereign-edge:~/honeypot/cowrie-etc$ nano cowrie.cfg
adminserver@sovereign-edge:~/honeypot/cowrie-etc$ cat cowrie.cfg
[honnyot]
hostname = srv-prod-database-01
adminserver@sovereign-edge:~/honeypot/cowrie-etc$ cd ..
adminserver@sovereign-edge:~/honeypot$ nano docker-compose.yml
adminserver@sovereign-edge:~/honeypot$ docker compose up -d
[+] up 1/1
✓Container cowrie Started 0.8s
adminserver@sovereign-edge:~/honeypot$
```

13.3. Configuración de Docker Compose.

Integramos la lógica en el archivo docker-compose.yml que se muestra en la segunda imagen. En este archivo ajustamos los tres puntos clave para que el sistema fuera útil y resiliente:

- Mapeo de puertos (22:2222): Es la parte central del engaño. El contenedor corre internamente en el puerto 2222, pero lo exponemos hacia el exterior a través del puerto estándar 22. De esta forma, cualquier escaneo de red detectará el servicio SSH habitual pero redirigirá el tráfico directamente al entorno controlado sin que el atacante sospeche que el sistema real está en otra ubicación.
- Persistencia de logs y captura de archivos: Para evitar la pérdida de datos tras un reinicio, vinculamos tres rutas locales con el contenedor. Esto nos asegura que tanto los logs de sesión como cualquier script o binario malicioso que el atacante intente descargar se almacenen de forma persistente en nuestra carpeta cowrie-downloads para su posterior análisis.
- Disponibilidad: Añadimos la configuración restart: always para garantizar que la trampa esté operativa 24/7 incluso si el proceso falla o el servidor se reinicia.



```
GNU nano 7.2 docker-compose.yml
services:
  cowrie:
    image: cowrie/cowrie:latest
    container_name: cowrie
    restart: always
    ports:
      - "22:2222"
    volumes:
      - ./cowrie-logs:/cowrie/cowrie-git/var/log/cowrie
      - ./cowrie-downloads:/cowrie/cowrie-git/var/lib/cowrie/downloads
      - ./cowrie-etc/cowrie.cfg:/cowrie/cowrie-git/etc/cowrie.cfg
```

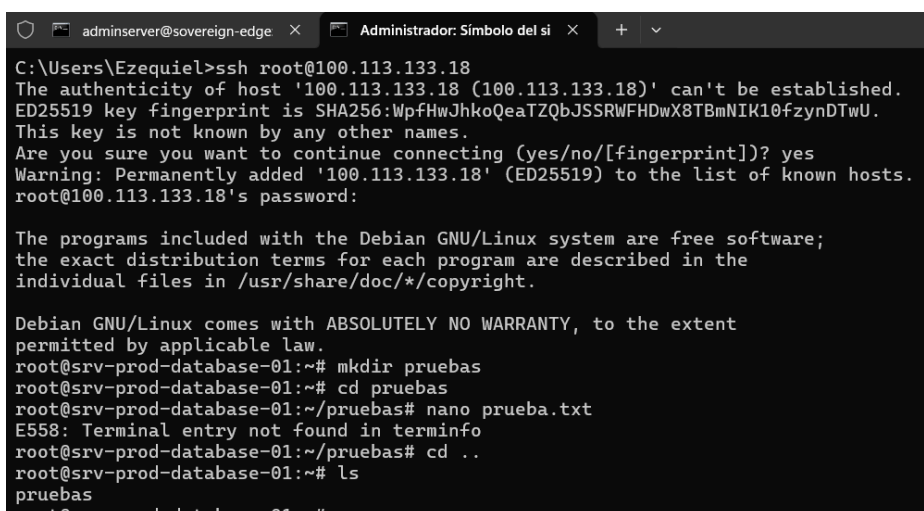
📄 docker-cowrie.yml

13.4. Prueba de intrusión y validación del entorno.

Para cerrar la fase de despliegue, realizamos una simulación de un ataque real desde una máquina externa. El objetivo era comprobar si un usuario malintencionado notaría algo extraño al entrar por el puerto estándar.

Como se muestra en la captura de la terminal, el resultado lo analizamos a continuación:

- Transparencia del engaño: Al ejecutar `ssh root@100.113.133.18`, la conexión se realizó por el puerto 22 de forma automática. El atacante entró directamente al contenedor de Cowrie sin recibir ninguna alerta lo que demuestra que nuestro mapeo de puertos en Docker funciona correctamente.
- Credibilidad de la identidad: Lo primero que aparece tras el login es el prompt `root@srv-prod-database-01`. Ver que el nombre que configuramos en el archivo `cowrie.cfg` se refleja perfectamente en la sesión; el atacante cree estar en un servidor crítico.
- Interacción y registro: En la imagen se ve cómo empezamos a simular un ataque con el sistema creando la carpeta pruebas y usando el comando `ls`. Aunque aparece un error típico de terminal al intentar abrir `nano`, el sistema simulado funciona como para que el atacante siga ejecutando comandos.
- Detección de inconsistencias: Un detalle interesante que observamos es que el banner de bienvenida indica Debian, a pesar de que nuestro servidor base es Ubuntu. Esta es una capa extra de confusión que añadimos para despistar a las herramientas de reconocimiento automático.

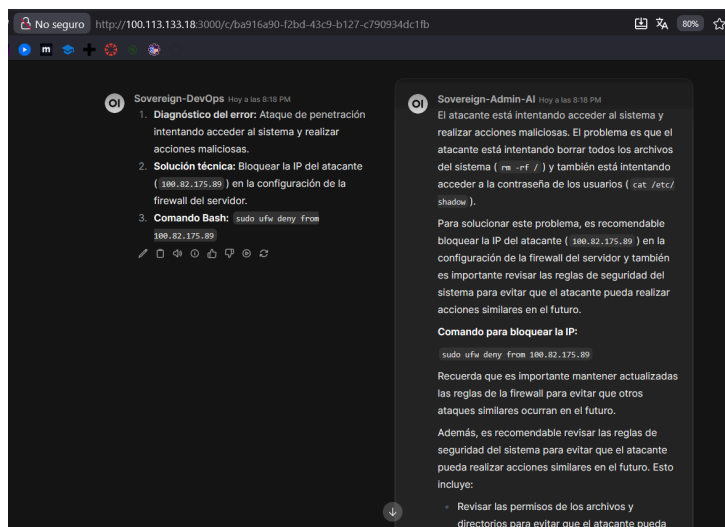


```
C:\Users\Ezequiel>ssh root@100.113.133.18
The authenticity of host '100.113.133.18 (100.113.133.18)' can't be established.
ED25519 key fingerprint is SHA256:WpfHwJhkoQeaTZQbJSSRWFHDwX8TBmNIK10fzynDTwU.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '100.113.133.18' (ED25519) to the list of known hosts.
root@100.113.133.18's password:

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
root@srv-prod-database-01:~# mkdir pruebas
root@srv-prod-database-01:~# cd pruebas
root@srv-prod-database-01:~/pruebas# nano prueba.txt
E558: Terminal entry not found in terminfo
root@srv-prod-database-01:~/pruebas# cd ..
root@srv-prod-database-01:~# ls
pruebas
root@srv-prod-database-01:~#
```


- Recomendaciones de endurecimiento: Más allá del bloqueo, la IA recomienda revisar los permisos de archivos y las reglas generales de seguridad para prevenir ataques similares en el futuro.



14. Hardening del sistema y Fail2Ban.

El objetivo de esta fase ha sido doble: configurar el firewall UFW para dejar pasar solo el tráfico estrictamente necesario y por otro, instalar Fail2Ban. Esta herramienta actúa como un "portero" automático si detecta que alguien intenta entrar por SSH al puerto real fallando la contraseña varias veces seguidas, banea esa IP al instante para frenar el ataque de fuerza bruta. Para confirmar que funciona, realizamos una prueba de bloqueo real intentando forzar el acceso nosotros mismos hasta quedar expulsados.

14.1. Análisis del estado del cortafuegos (UFW).

Al ejecutar `sudo ufw status verbose`, confirmamos que el servidor está blindado bajo una política de bloqueo total por defecto. La clave de esta configuración es que hemos separado el tráfico para no correr riesgos: el puerto 22 se deja abierto a propósito para que los atacantes piquen en el Honeypot, mientras que nosotros usamos el 2222 y la red privada de Tailscale para la gestión administrativa real.

Además, hemos habilitado el puerto 3000 para que la interfaz de la IA funcione correctamente y nos hemos asegurado de que todas las reglas cubren tanto IPv4 como IPv6. De esta forma, el servidor solo responde a lo justo y necesario.

```
adminserver@sovereign-edge:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), deny (routed)
New profiles: skip

To Action From
--
3000/tcp ALLOW IN Anywhere
22/tcp ALLOW IN Anywhere
Anywhere on tailscale0 ALLOW IN Anywhere
2222/tcp ALLOW IN Anywhere
3000/tcp (v6) ALLOW IN Anywhere (v6)
22/tcp (v6) ALLOW IN Anywhere (v6)
Anywhere (v6) on tailscale0 ALLOW IN Anywhere (v6)
2222/tcp (v6) ALLOW IN Anywhere (v6)
```

15. Instalación y puesta en marcha de Fail2Ban.

Para activar la defensa automática contra ataques de fuerza bruta, instalamos Fail2Ban. Como se muestra en la captura, el proceso descarga e instala el servicio junto a las dependencias necesarias para su funcionamiento.

```
adminserver@sovereign-edge:~$ sudo apt install fail2ban
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes adicionales:
 python3-pyasyncore python3-pyinotify whois
Paquetes sugeridos:
 mailx monit sqlite3 python-pyinotify-doc
Se instalarán los siguientes paquetes NUEVOS:
 fail2ban python3-pyasyncore python3-pyinotify whois
0 actualizados, 4 nuevos se instalarán, 0 para eliminar y 0 no actualizados.
Se necesita descargar 496 kB de archivos.
Se utilizarán 2.572 kB de espacio de disco adicional después de esta operación.
¿Desea continuar? [S/n] s
Des:1 http://es.archive.ubuntu.com/ubuntu noble/main amd64 python3-pyasyncore all 1.0.2-2 [10,1 kB]
Des:2 http://es.archive.ubuntu.com/ubuntu noble-updates/universe amd64 fail2ban all 1.0.2-3ubuntu0.1 [409 kB]
Des:3 http://es.archive.ubuntu.com/ubuntu noble/main amd64 python3-pyinotify all 0.9.6-2ubuntu1 [25,0 kB]
Des:4 http://es.archive.ubuntu.com/ubuntu noble/main amd64 whois amd64 5.5.22 [51,7 kB]
Descargados 496 kB en 0s (4.343 kB/s)
```

15.1. Protección de la configuración original.

Como se muestra en la captura, realizamos una copia del archivo de configuración mediante el comando `sudo cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local`.

El objetivo de este paso es conservar el archivo original intacto y trabajar sobre la copia .local. De esta forma, nos aseguramos de que nuestras reglas personalizadas no se sobrescriban si el servicio se actualiza en el futuro manteniendo siempre una copia de seguridad de los ajustes de fábrica por si necesitáramos consultarlos o restaurarlos.

```
adminserver@sovereign-edge:~$ sudo cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local
adminserver@sovereign-edge:~$
```

15.2. Configuración de reglas y baneo progresivo.

Tras asegurar el archivo original, editamos el nuevo jail.local para definir cómo debe responder el servidor ante un ataque. Como se ve en la captura, nos centramos en la sección [sshd] para activar la monitorización del acceso SSH.

Para ello, activamos el servicio con `enabled = true` y fijamos el límite en 5 reintentos (`maxretry`). Si alguien supera este número en un tiempo de 10 minutos (`findtime`), el sistema banea la IP automáticamente durante otros 10 minutos (`bantime`). Además, como se observa en la parte inferior de la imagen, hemos habilitado el autoincremento de baneo (`bantime.increment`). Si un atacante vuelve a intentar entrar justo después de que expire su baneo, el sistema le multiplica el tiempo de bloqueo de forma progresiva. Así evitamos que los bots se queden en un bucle infinito atacando cada vez que se les levanta la sanción.

```
adminserver@sovereign-edge:~$ sudo nano /etc/fail2ban/jail.local
adminserver@sovereign-edge:~$ sudo grep -A 10 "[sshd]" /etc/fail2ban/jail.local
[sshd]

enabled = true
port = 2222
filter = sshd
logpath = /var/log/auth.log
maxretry = 3
bantime = 3600
findtime = 600
bantime.increment = true
```

 jail.local

15.3. Verificación de la activación de Fail2Ban.

Tras ejecutar el comando `sudo fail2ban-client status sshd`, verificamos que el servicio se ha levantado correctamente con las reglas que definimos. Este comando es fundamental para revisar en tiempo real la eficacia de nuestra defensa.

Como se observa en el panel de estado, el sistema ya está monitorizando el archivo de registros `/var/log/auth.log` en busca de intentos de acceso fallidos. Ahora mismo los contadores de "Currently banned" y "Total banned" aparecen a cero, lo que confirma que el servicio está activo y a la espera de interceptar cualquier ataque.

```
adminserver@sovereign-edge:~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
|   |- Currently failed: 0
|   |- Total failed:     0
|   `-- File list:      /var/log/auth.log
`- Actions
    |- Currently banned: 0
    |- Total banned:     0
    `-- Banned IP list:
```

15.4. Prueba de funcionamiento y bloqueo de seguridad.

Para comprobar que el sistema realmente responde, hicimos una prueba simulando un ataque de fuerza bruta. Como se ve en la primera captura, intentamos entrar por SSH al puerto 2222 fallando la contraseña a propósito. Al llegar al tercer intento (que es el límite que configuramos), el servidor nos cortó la conexión y nos expulsó automáticamente.

En la segunda imagen se ve que el bloqueo ha hecho efecto: al intentar conectar de nuevo, ya ni siquiera nos pide la clave, directamente nos da un error de "Connection timed out". Esto nos confirma que Fail2Ban ha detectado los intentos fallidos y ha pasado nuestra IP al firewall para bloquearnos el acceso por completo. Con esto validamos que el servidor ya sabe defenderse solo y cierra el paso a cualquier intruso sin que tengamos que intervenir en ningún momento.

```
C:\Users\Oleg>ssh -p 2222 adminserver@100.113.133.18
adminserver@100.113.133.18's password:
Permission denied, please try again.
adminserver@100.113.133.18's password:
Permission denied, please try again.
adminserver@100.113.133.18's password:
adminserver@100.113.133.18: Permission denied (publickey,password).
```

```
C:\Users\Oleg>ssh -p 2222 adminserver@100.113.133.18
ssh: connect to host 100.113.133.18 port 2222: Connection timed out
```

15.5. Verificación de resultados y baneo de la IP.

Para cerrar la prueba, volvimos a consultar el estado del servicio con el comando `sudo fail2ban-client status sshd`. Como se puede ver en la captura, ahora sí refleja la actividad del ataque simulado que realizamos anteriormente.

En el apartado de "Filter", aparece que se han detectado los 3 fallos de inicio de sesión (Total failed: 3). Lo más importante es lo que vemos en "Actions": el sistema ha pasado de tener los contadores a cero a mostrar un baneo activo. En la "Banned IP list" aparece la dirección IP desde la que intentamos forzar el acceso, confirmando que Fail2Ban la ha identificado correctamente y la ha expulsado del sistema.

```
adminserver@sovereign-edge:~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
| |- Currently failed: 0
| |- Total failed:    3
| `-- File list:      /var/log/auth.log
`- Actions
   |- Currently banned: 1
   |- Total banned:    1
   `-- Banned IP list: 100.109.31.17
adminserver@sovereign-edge:~$ |
```

15.6. Análisis de registros de autenticación (auth.log).

Revisamos el archivo `/var/log/auth.log`, donde el servidor registra cada intento de acceso. En la captura se ve la secuencia del ataque simulado: aparecen los tres mensajes de "Failed password for adminserver" de la misma IP.

Tras el tercer fallo, el sistema registra un "Connection reset". Esto es la prueba de que justo en ese momento, el servicio de Fail2ban cortó la comunicación con el atacante. Ver estos logs nos da la tranquilidad y certeza de que el servidor no solo está bloqueando los accesos sino que está dejando un registro claro de quién ha intentado entrar, desde qué IP y en qué momento exacto se le denegó el paso.

```
2026-04-25T20:58:03.999342+00:00 sovereign-edge sshd[8573]: Failed password for adminserver from 100.109.31.17 port 5215
0 ssh2
2026-04-25T20:58:09.420847+00:00 sovereign-edge sshd[8573]: Failed password for adminserver from 100.109.31.17 port 5215
0 ssh2
2026-04-25T20:58:14.055929+00:00 sovereign-edge sshd[8573]: Failed password for adminserver from 100.109.31.17 port 5215
0 ssh2
2026-04-25T20:58:14.491536+00:00 sovereign-edge sshd[8573]: Connection reset by authenticating user adminserver 100.109.
31.17 port 52150 [preauth]
2026-04-25T20:58:14.491634+00:00 sovereign-edge sshd[8573]: PAM 2 more authentication failures; logname= uid=0 euid=0 tt
y=ssh ruser= rhost=100.109.31.17 user=adminserver
2026-04-25T21:00:06.062971+00:00 sovereign-edge sudo: adminserver : TTY=pts/1 ; PWD=/home/adminserver ; USER=root ; COMM
```

 auth.log  auth.log.1

15.7. Desbloqueo manual de IPs.

Para finalizar las comprobaciones y recuperar la conectividad desde nuestro equipo de pruebas, ejecutamos el comando `sudo fail2ban-client set sshd unbanip 100.109.31.17`. Este paso es necesario para eliminar manualmente nuestra IP de la lista negra sin tener que esperar a que expire el tiempo de castigo que se ha configurado.

Como se ve en la captura, el terminal devuelve un "1", lo que indica que se ha realizado con éxito y una dirección IP ha sido liberada. Con esto, confirmamos que además de tener una defensa automática, mantenemos el control total sobre el servicio pudiendo gestionar excepciones o corregir bloqueos accidentales de forma inmediata.

```
adminserver@sovereign-edge:~$ sudo fail2ban-client set sshd unbanip 100.109.31.17
1
adminserver@sovereign-edge:~$ |
```

15.8. Análisis del log específico de Fail2Ban.

Para terminar de auditar el comportamiento del sistema configurado, revisamos el archivo `/var/log/fail2ban.log`. A diferencia del log anterior, este se centra en las decisiones que hace la herramienta. Como se ve en la captura, al principio aparecen las reglas que configuramos: el findtime de 600 segundos, el bantime de una hora y la activación del incremento de baneo.

En la parte inferior de la imagen se puede ver el momento exacto del ataque simulado. El sistema registra tres líneas con el mensaje "Found IP" que corresponden a cada uno de nuestros intentos fallidos. En cuanto ha detectado el tercero, se genera automáticamente la línea de "Ban IP". Esta es la prueba definitiva de que Fail2Ban está actuando correctamente guardando la información en el log de autenticación y ejecutando la orden de bloqueo en el firewall de inmediato.

```
adminserver@sovereign-edge:~/honeypot$ sudo tail -f /var/log/fail2ban.log
2026-04-25 21:25:19,446 fail2ban.filter [10704]: INFO findtime: 600
2026-04-25 21:25:19,446 fail2ban.actions [10704]: INFO banTime: 3600
2026-04-25 21:25:19,446 fail2ban.jail [10704]: INFO Set banTime.increment = True
2026-04-25 21:25:19,446 fail2ban.filter [10704]: INFO encoding: UTF-8
2026-04-25 21:25:19,447 fail2ban.filter [10704]: INFO Added logfile: '/var/log/auth.log' (pos = 32567, hash =
1415d441589b117f2f1057d2b68bd7d66cbac27)
2026-04-25 21:25:19,448 fail2ban.jail [10704]: INFO Jail 'sshd' started
2026-04-25 21:52:18,789 fail2ban.filter [10704]: INFO [sshd] Found 100.109.31.17 - 2026-04-25 21:52:18
2026-04-25 21:52:24,054 fail2ban.filter [10704]: INFO [sshd] Found 100.109.31.17 - 2026-04-25 21:52:24
2026-04-25 21:52:25,658 fail2ban.filter [10704]: INFO [sshd] Found 100.109.31.17 - 2026-04-25 21:52:25
2026-04-25 21:52:26,246 fail2ban.actions [10704]: NOTICE [sshd] Ban 100.109.31.17
```

📄 fail2ban.log 📄 fail2ban.log.1

16. Instalación de Nginx Proxy Inverso.

Se procede a realizar la instalación de Nginx en nuestro servidor de manera que actúe como proxy inverso, se ha optado por el mismo debido a que con los pocos recursos que utiliza y su valoración en servidores como el nuestro e incluso servidores dedicados especialmente a servicios web:

```
adminserver@sovereign-edge:~$ sudo apt install nginx -y
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes adicionales:
  nginx-common
Paquetes sugeridos:
  fcgiwrap nginx-doc ssl-cert
Se instalarán los siguientes paquetes NUEVOS:
  nginx nginx-common
0 actualizados, 2 nuevos se instalarán, 0 para eliminar y 77 no actualizados.
Se necesita descargar 565 kB de archivos.
Se utilizarán 1.596 kB de espacio de disco adicional después de esta operación.
Des:1 http://es.archive.ubuntu.com/ubuntu noble-updates/main amd64 nginx-common all 1.24.0-2ubuntu7.6 [43,5 kB]
```

Una vez realizado la instalación procedemos a verificar el estado del servicio y que se encuentre operativo en nuestro servidor, para seguidamente proceder a configurar los parámetros a tener en cuenta y que funcione correctamente y debidamente en nuestro servidor:

```
adminserver@sovereign-edge:~$ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Active: active (running) since Sat 2026-04-25 21:00:31 UTC; 5min ago
     Docs: man:nginx(8)
  Process: 9284 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
  Process: 9285 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
 Main PID: 9318 (nginx)
    Tasks: 5 (limit: 9375)
   Memory: 3.7M (peak: 8.4M)
      CPU: 18ms
   CGroup: /system.slice/nginx.service
           └─9318 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             └─9321 "nginx: worker process"
               └─9322 "nginx: worker process"
                 └─9323 "nginx: worker process"
                   └─9324 "nginx: worker process"

abr 25 21:00:31 sovereign-edge systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy s>
abr 25 21:00:31 sovereign-edge systemd[1]: Started nginx.service - A high performance web server and a reverse proxy se>
```

17. Creación de un certificado SSL autofirmado.

A continuación creamos un directorio, donde almacenaremos en este caso el certificado autofirmado de nuestro servidor para el uso de https en nuestra interfaz web de nuestra IA local, una vez creado el directorio procedemos a generar el certificado mediante openssl, con lo cual hemos asignado varios parámetros como: País, Localidad, Organización, también la clave privada y el certificado de los cuales se almacenan en el sistema.

[illegible]

17.1. Configuración de Nginx como proxy inverso con HTTPS.

En el siguiente archivo de configuración ubicado en el directorio mencionado a continuación `/etc/nginx/sites-available/sovereign-ia`, donde hemos realizado la configuración en dos bloques, a continuación podemos observar la configuración de Nginx para que actúe como proxy inverso y seguro en nuestra pagina web:

Redirección HTTP a HTTPS:

- Escucha en el puerto 80.
- Redirige automáticamente todo el tráfico a HTTPS (puerto 443) mediante una redirección 301 permanente.

Servidor HTTPS con proxy inverso:

- Escucha en el puerto 443 con SSL activo.
- Utiliza el certificado y la clave SSL previamente creados.
- Define protocolos TLS seguros.
- Reenvía las peticiones al servicio interno que corre en <http://localhost:3000>.

```
adminserver@sovereign-edge:~$ cat /etc/nginx/sites-available/sovereign-ia

server {
    listen 80;
    server_name 100.113.133.18;
    return 301 https://$host$request_uri;
}

#Proxy inverso
server {
    listen 443 ssl;
    server_name 100.113.133.18;

    ssl_certificate      /etc/nginx/ssl/selfsigned.crt;
    ssl_certificate_key  /etc/nginx/ssl/selfsigned.key;

    ssl_protocols        TLSv1.2 TLSv1.3;
    ssl_ciphers           HIGH:!aNULL:!MD5;

    location / {
        proxy_pass        http://localhost:3000;
        proxy_http_version 1.1;

        proxy_set_header  Upgrade $http_upgrade;
        proxy_set_header  Connection 'upgrade';
        proxy_set_header  Host $host;
        proxy_set_header  X-Real-IP $remote_addr;
        proxy_cache_bypass $http_upgrade;
    }
}

adminserver@sovereign-edge:~$ |
```

sovereign-ia

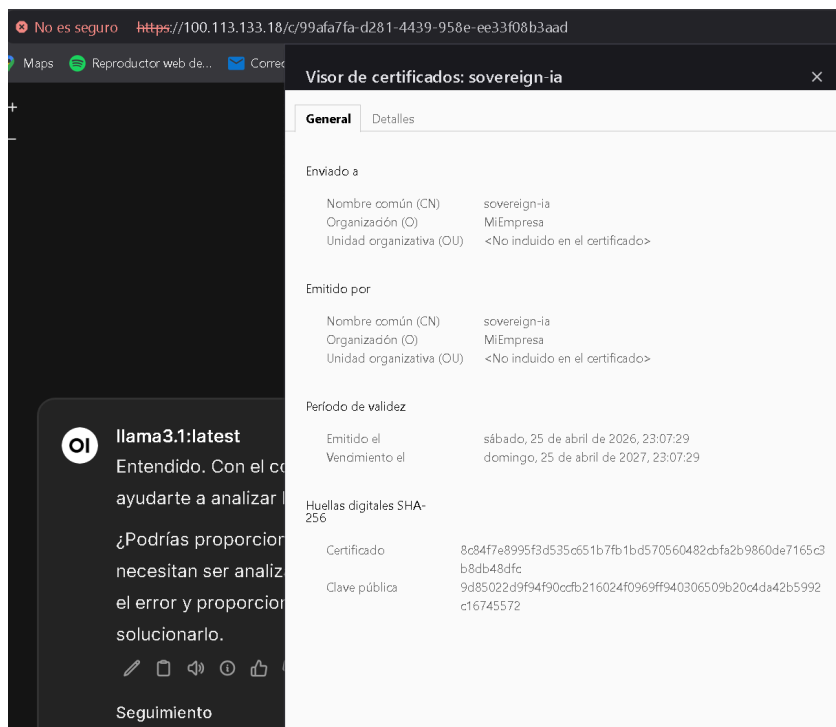
El sistema implementando corresponde a un servidor de IA de uso interno, destinado al análisis de amenazas capturadas por un honeypot. La IA y su interfaz web no están expuestas públicamente, siendo accesibles únicamente por nosotros.

Nginx se utiliza como proxy inverso y punto de acceso interno, permitiendo centralizar la gestión HTTPS y actuar como intermediario entre las personas que gestionan la IA (en este caso nosotros) y la aplicación de IA. Mediante la configuración de cabeceras HTTP como Host, X-Real-IP, Upgrade y Connection, se garantiza que la interfaz web funcione correctamente, que se registre la IP real de acceso para tareas de auditoría y que se soporten conexiones persistentes necesarias para el análisis en tiempo real de lo que pueda ocurrir en la IA.

17.2. Verificación del estado del certificado y cifrado de la conexión.

La figura muestra el certificado SSL utilizado para asegurar el acceso a la interfaz web del servidor de IA. Se trata de un certificado autofirmado, que ha sido generado internamente como se mostró en apartados anteriores, por lo que el navegador nos indica que la conexión no es segura al no estar firmado por una autoridad certificadora pública.

A pesar de esta advertencia, la comunicación se realiza mediante HTTPS, garantizando el cifrado de los datos transmitidos entre el cliente y el servidor como se puede observar en la imagen.



17.3. Implementación de notificaciones al canal de Discord sobre intentos de acceso.

En nuestro script de Python procedemos a establecer en enlace llamado WEBHOOK de nuestro canal de discord lo cual nos permite que una vez se ejecute el script y analice si existen nuevos intentos de acceso, se puede observar que realizará el análisis de los logs en la ruta donde se encuentra los logs de nuestro honeypot y se puede visualizar en la siguiente imagen:

```
adminserver@sovereign-edge:~/honeypot$ cat vincular_ia.py
import json
import subprocess
import os
import time

# CONFIGURACIÓN
DISCORD_WEBHOOK = "https://discord.com/api/webhooks/1497713671638614037/Ieuj9v6HhFTvdu5Gu0c7R41-UtLow58Z9Kq4ozs91Ygl-ir9dYynR3p5dAJq81XwffAZ"
LOG_FILE = os.path.expanduser('~/.honeypot/cowrie-logs/cowrie.json')
ESTADO_FILE = os.path.expanduser('~/.honeypot/ultimo_procesado.txt')

def cargar_ultima_linea():
    if os.path.exists(ESTADO_FILE):
        with open(ESTADO_FILE, 'r') as f:
            return int(f.read().strip() or 0)
    return 0

def guardar_ultima_linea(n):
    with open(ESTADO_FILE, 'w') as f:
        f.write(str(n))

def enviar_discord(mensaje):
    try:
        requests.post(DISCORD_WEBHOOK, json={"content": mensaje})
    except Exception as e:
        print(f"Error enviando a Discord: {e}")

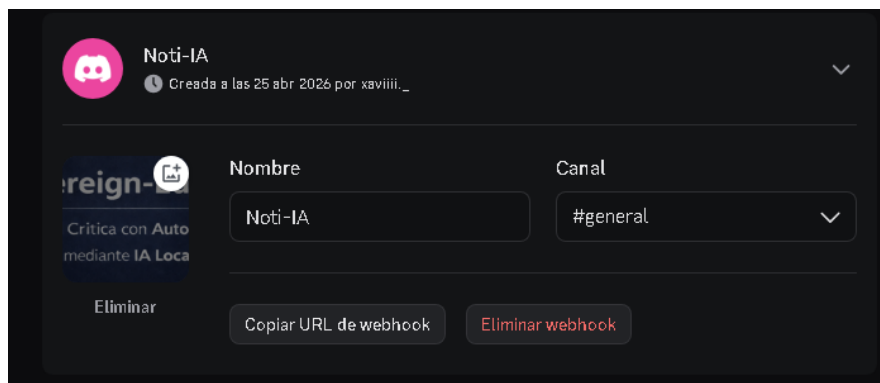
def analizar_con_ia(ip, usuario, password):
```

análisis IA.py

Procedemos a ejecutar el script y comience el análisis de los logs y nos envíe las alertas a nuestro canal de discord:

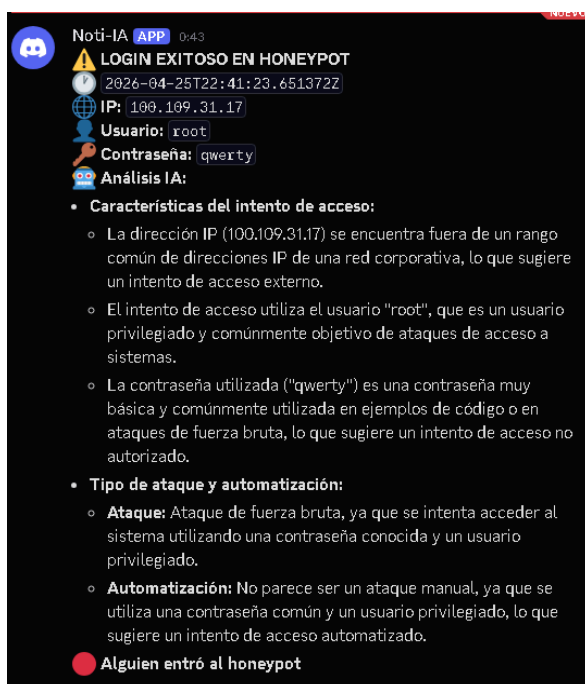
```
adminserver@sovereign-edge:~/honeypot$ python3 ~/honeypot/vincular_ia.py
🔍 Monitorizando honeypot...
✅ Procesadas 6 líneas nuevas.
adminserver@sovereign-edge:~/honeypot$ |
```

En nuestro canal de discord procedemos a editar las opciones del canal las cuales nos permite copiar la URL de Webhook, que es lo que nos permite realizar este tipo de alertas:



Anteriormente ejecutamos el script y después del análisis recopilado en nuestro servidor de los intentos fallidos de acceso al honeypot, tenemos como mensaje de alerta generado automáticamente por cada acceso exitoso o fallido como se muestra en la imagen, el mensaje nos muestra varios parámetros con los cuales se realizó el acceso:

- Fecha y hora
- IP desde donde se intentó el acceso
- Usuario con el que se intentó el acceso
- Contraseña
- Análisis de nuestro Modelo de IA del acceso o intención del mismo.



18. Base de datos de logs (Elasticsearch y Filebeat) y Dashboard de Kibana: Visualización de amenazas.

Para que todos los ataques del Honeypot y los eventos del sistema queden registrados, hemos montado una base de datos centralizada que gestione esa información. Como se ve en la imagen, empezamos configurando los archivos docker-compose.yml y filebeat.yml para definir cómo Elasticsearch guardará los datos y cómo Filebeat los recolectarán.

Además, realizamos un ajuste en el kernel con el comando `vm.max_map_count`. Linux viene con un límite de memoria demasiado bajo para lo que necesita Elasticsearch lo que causaría errores de estabilidad. Al subir este valor a 262144 y dejarlo configurado en el archivo `sysctl.conf`, nos aseguramos de que la base de datos pueda procesar miles de líneas de logs en tiempo real sin fallar el sistema, dejando todo listo para que se visualice correctamente en Kibana.

```
adminserver@sovereign-edge:~/honeypot/elk$ nano docker-compose.yml
adminserver@sovereign-edge:~/honeypot/elk$ nano filebeat.yml
adminserver@sovereign-edge:~/honeypot/elk$ chmod 644 filebeat.yml
adminserver@sovereign-edge:~/honeypot/elk$ sudo sysctl -w vm.max_map_count=262144
vm.max_map_count = 262144
adminserver@sovereign-edge:~/honeypot/elk$ echo "vm.max_map_count=262144" | sudo tee -a /etc/sysctl.conf
vm.max_map_count=262144
```

18.1. Configuración del fichero Docker Compose.

En esta captura se muestra el archivo `docker-compose.yml`. Aquí hemos definido cómo deben levantarse y comunicarse los servicios de Elasticsearch, Kibana y Filebeat. Lo primero es la versión de las imágenes (8.13.4), asegurando que todo los servicios sean compatibles entre sí.

En la configuración de Elasticsearch, hemos limitado el uso de memoria con la variable `ES_JAVA_OPTS` para no saturar el servidor y hemos mapeado el puerto 9200, que es por donde recibirá los logs. Para Kibana definimos el puerto 5601 que es la interfaz web desde la cual visualizamos los datos. Por último, en la sección de Filebeat se puede ver la configuración de los volúmenes que le damos permiso al contenedor para leer las rutas del sistema anfitrión donde se guardan los logs del Honeypot.

```
adminserver@sovereign-edge:~/honeypot/elk$ cat docker-compose.yml
services:

  elasticsearch:
    image: docker.elastic.co/elasticsearch/elasticsearch:8.13.4
    container_name: elasticsearch

    environment:
      - discovery.type=single-node
      - xpack.security.enabled=false
      - ES_JAVA_OPTS=-Xms512m -Xmx512m

    ports:
      - "9200:9200"

    volumes:
      - esdata:/usr/share/elasticsearch/data

    restart: unless-stopped

  kibana:
    image: docker.elastic.co/kibana/kibana:8.13.4
    container_name: kibana

    environment:
```

Se puede observar la configuración completa en `docker-kibana,elk,filebeat.yml`.

18.2. Puesta en marcha de la infraestructura de análisis mediante contenedores.

Tras haber ajustado la configuración del fichero docker-compose.yml, procedemos al despliegue real de la arquitectura. En la captura se observa el uso del comando `docker compose up -d`, que se encarga de descargar las imágenes oficiales de Elasticsearch, Kibana y Filebeat en su versión 8.13.4.

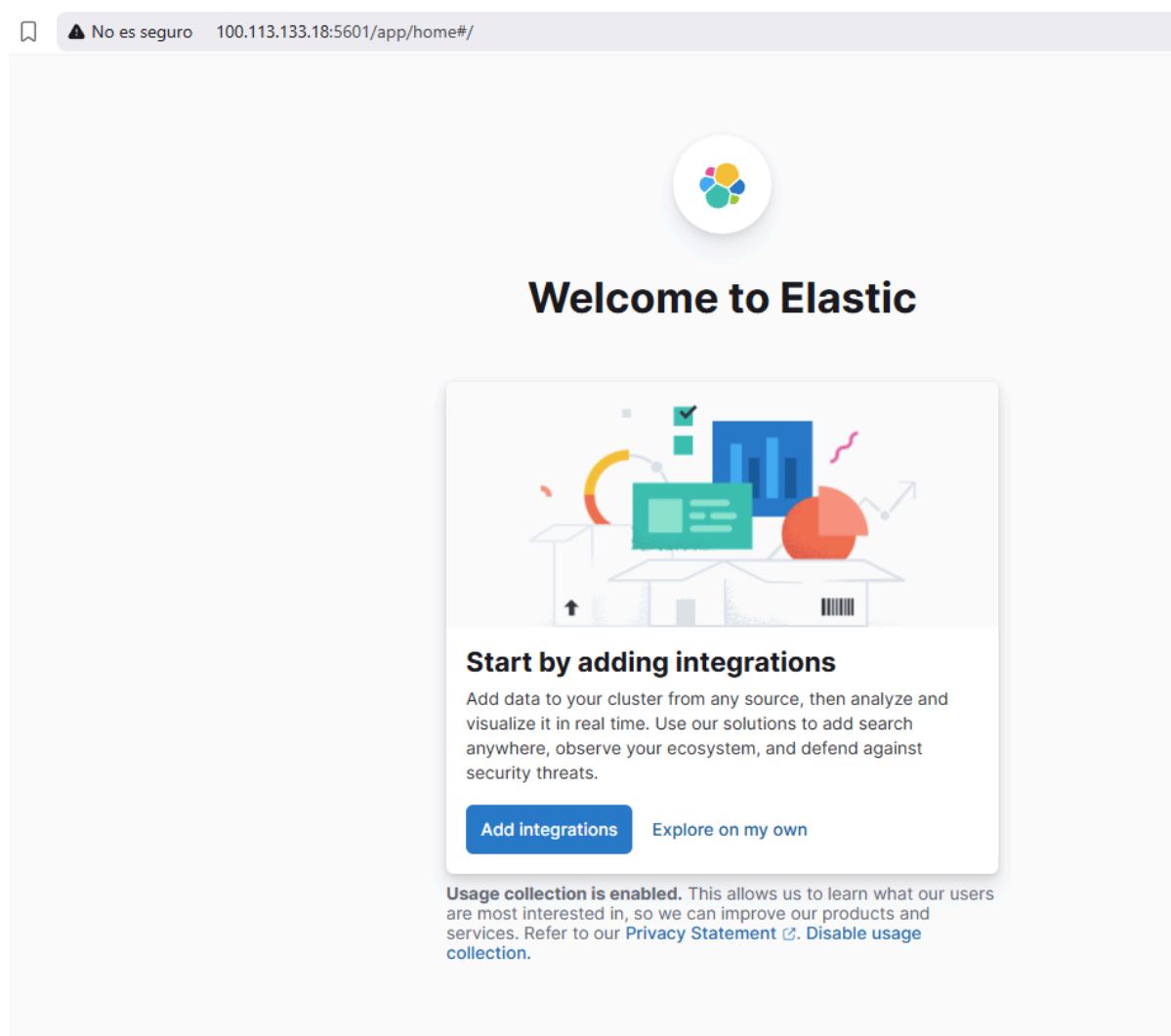
Como se ve en la imagen, el sistema no solo arranca los contenedores sino que crea automáticamente una red aislada (`elk_default`) para que el tráfico de los registros no interfiera con otros servicios del servidor. La verificación de todos los componentes en estado "Started" nos da la seguridad de que la base de datos ya está lista para recibir, procesar y mostrar visualmente cada ataque interceptado por el Honeypot, aumentando así la capacidad de monitorización del proyecto.

```
adminserver@sovereign-edge:~/honeypot/elk$ docker compose up -d
[+] up 40/40
✓Image docker.elastic.co/elasticsearch/elasticsearch:8.13.4 Pulled 70.1s
✓Image docker.elastic.co/kibana/kibana:8.13.4 Pulled 61.3s
✓Image docker.elastic.co/beats/filebeat:8.13.4 Pulled 18.0s
✓Network elk_default Created 0.0s
✓Volume elk_esdata Created 0.0s
✓Container elasticsearch Started 0.6s
✓Container filebeat Started 0.4s
✓Container kibana Started 0.4s
adminserver@sovereign-edge:~/honeypot/elk$
```


18.3. Acceso a la interfaz de gestión y visualización en Kibana.

Una vez desplegados los contenedores, validamos el funcionamiento accediendo a la interfaz web de Kibana. Como se aprecia en la captura, hemos entrado a través de la dirección IP del servidor por el puerto 5601, visualizando la pantalla de bienvenida de Elastic. Este es el panel de control desde el cual transformaremos los datos de los ataques en gráficos y mapas interactivos.

El hecho de que la página cargue confirma que la comunicación entre Kibana y Elasticsearch es correcta. El sistema está listo para que empecemos a filtrar los registros recolectados por Filebeat.



18.4. Configuración de alias en Kibana.

Para que Kibana pueda interpretar los datos almacenados en Elasticsearch es necesario definir un Index Pattern. Como se observa en la captura, hemos creado el patrón `cowrie-attacks-*` que actúa como un alias para agrupar todos los índices que contienen los registros del Honeypot, así permite realizar búsquedas unificadas sobre diferentes archivos de logs históricos sin tener que consultarlos uno por uno.

En la imagen se ve cómo Kibana ha identificado automáticamente los campos disponibles, asignándoles tipos de datos específicos como `date` para la marca de tiempo o `keyword` para las direcciones IP y nombres de usuario. Al marcar estos campos como "searchable" y "aggregatable" estamos habilitando la capacidad de filtrar ataques en tiempo real y así construir el dashboard de seguridad del proyecto posteriormente.

cowrie-attacks-* Delete Edit

Index pattern: `cowrie-attacks-*` Time field: `@timestamp` Default

Fields (7304) **Scripted fields (0)** **Field filters (0)** **Relationships (0)**

Search

Field type: 15 Schema type: 2 Refresh Add field

Name ↑	Type	Format	Searchable	Aggregatable	Excluded
<code>@timestamp</code>	date		•	•	Edit
<code>._id</code>	._id		•		Edit
<code>._index</code>	._index		•	•	Edit
<code>._score</code>					Edit
<code>._source</code>	._source				Edit
<code>activemq.caller</code>	keyword		•	•	Edit
<code>activemq.log.stack_trace</code>	keyword		•	•	Edit
<code>activemq.thread</code>	keyword		•	•	Edit
<code>activemq.user</code>	keyword		•	•	Edit
<code>agent.build.original</code>	keyword		•	•	Edit

Rows per page: 10 < 1 2 3 4 5 ... 731 >

18.5. Monitorización activa y análisis de registros en tiempo real.

Para verificar que todo está operativo utilizamos la herramienta Discover de Kibana. Como se observa en la captura, el sistema ya está recibiendo y procesando datos en tiempo real del Honeypot. El gráfico histórico muestra la distribución temporal, permitiéndonos identificar picos de actividad que podrían ser intentos de intrusión masivos o escaneos de vulnerabilidades.

Podemos ver campos críticos como el `agent.hostname`, la versión del servicio y el message detallado de cada evento como inicios de sesión fallidos o cierres de conexión. Esto nos confirma que los datos funcionan sin errores: el Honeypot genera el log, Filebeat lo captura, Elasticsearch lo indexa y Kibana lo muestra de forma gráfica y fácil, así nos demuestra la transformación del tráfico de red en gráficos y mensajes más sencillos de administrar que leer simplemente logs en la terminal.



18.6. Configuración del contenedor de Filebeat.

En esta captura se puede ver el contenido del archivo filebeat.yml que gestiona el flujo de datos desde el Honeypot hasta la base de datos. Se observa cómo hemos configurado el agente para que monitorice específicamente el archivo cowrie.json. Al utilizar el parser ndjson garantizamos que Filebeat interprete directamente los registros del Honeypot para su posterior indexación.

Hemos implementado una regla de renombramiento para el campo src_ip a [source.ip](#). En la sección de output, hemos desactivado el ciclo de vida de índices (ilm.enabled: false) para tener un control sobre el nombre de los índices que se organizarán por fecha siguiendo el patrón cowrie-attacks-%{+yyyy.MM.dd}. Nos garantiza que la información esté ordenada y lista para ser consultada desde la interfaz de Kibana.

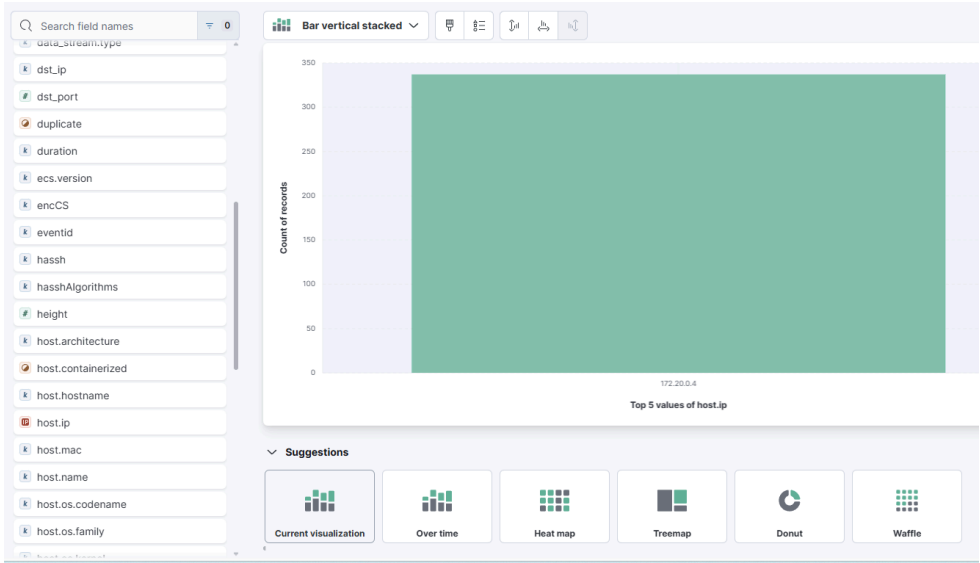
Se puede verificar la configuración completa en `filebeat.yml`.

18.7. Configuración de visualizaciones y analítica en Kibana Lens.

Una vez que los datos se han indexado, utilizamos la herramienta Lens de Kibana para transformar los registros en formato visual. Como se observa en la imagen, el sistema nos permite arrastrar los campos recolectados por Filebeat para crear gráficos de forma intuitiva. En este caso, hemos creado un gráfico de barras que analiza los registros basándose en el campo host.ip.

Esto es importante para el proyecto ya que nos permite monitorizar y detectar anomalías en el tráfico. Kibana nos ofrece sugerencias inteligentes como mapas de calor o gráficos circulares para representar la información de la manera que mejor nos convenga. Al tener parámetros tan variados como puertos, nombres de usuario o comandos ejecutados podemos construir

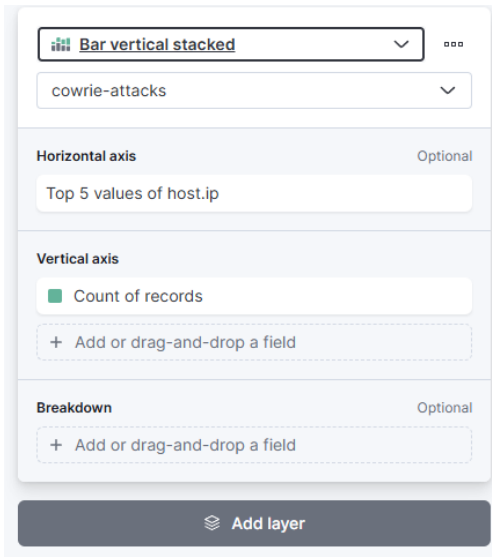
dashboards personalizados que nos facilite el análisis forense y la toma de decisiones ante posibles ataques.



18.8. Personalización de métricas y filtros en Kibana Lens.

En esta captura se detalla el panel de configuración de la visualización donde definimos los criterios para el análisis de los datos. Como se observa, estamos trabajando sobre el índice de ataques del Honeypot y lo hemos configurado para mostrar los 5 valores más frecuentes del campo `host.ip` que nos permite analizar cuáles son los orígenes más activos en el envío de tráfico.

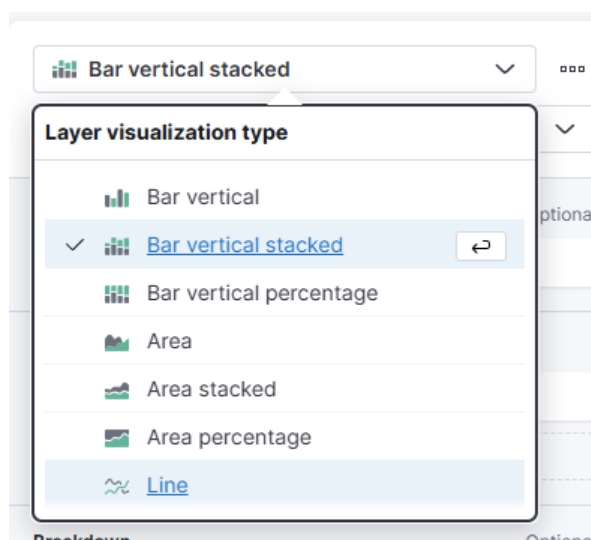
En el eje vertical, hemos establecido un recuento de registros lo que nos da un volumen de actividad por cada IP detectada.



18.9. Selección de modelos de representación visual en Kibana.

En esta fase del diseño del dashboard consultamos las diferentes opciones que ofrece Kibana para representar los datos recolectados. Como se observa en el menú de la captura, el sistema permite elegir entre múltiples tipos de visualización según el objetivo. Disponemos de opciones como barras verticales, gráficos de área y gráficos de líneas.

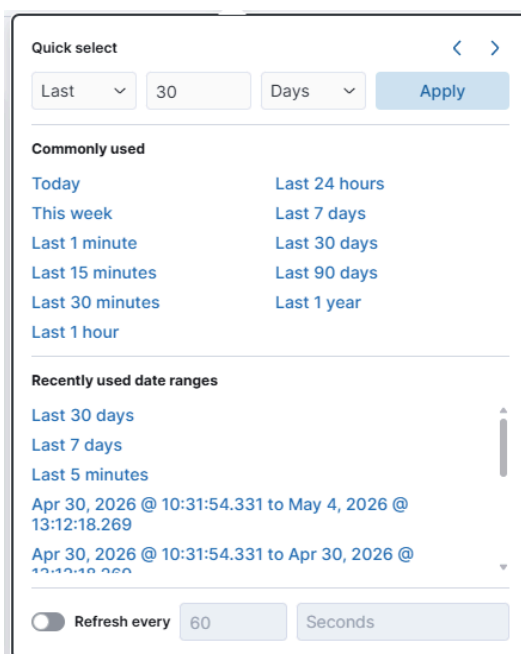
Este ajuste es clave para el proyecto ya que no todos los datos se interpretan igual. La posibilidad de cambiar el tipo de gráfico permite adaptarnos para resaltar la información más crítica en cada momento.



18.10. Gestión de rangos temporales para el análisis de incidentes.

Como se observa en la captura, Kibana ofrece un rango temporal que permite filtrar los logs desde los últimos minutos hasta el último año. Esta función nos permite hacer tanto una monitorización en tiempo real hasta un análisis histórico a largo plazo para identificar ataques.

En la imagen también se aprecia la opción de "Refresh every" para que el dashboard se actualice automáticamente cada ciertos segundos. Esto nos permite que el panel funcione como un monitor de seguridad en directo y así saltar rápidamente de una visión general del estado a un análisis detallado de un evento ocurrido en un momento específico.

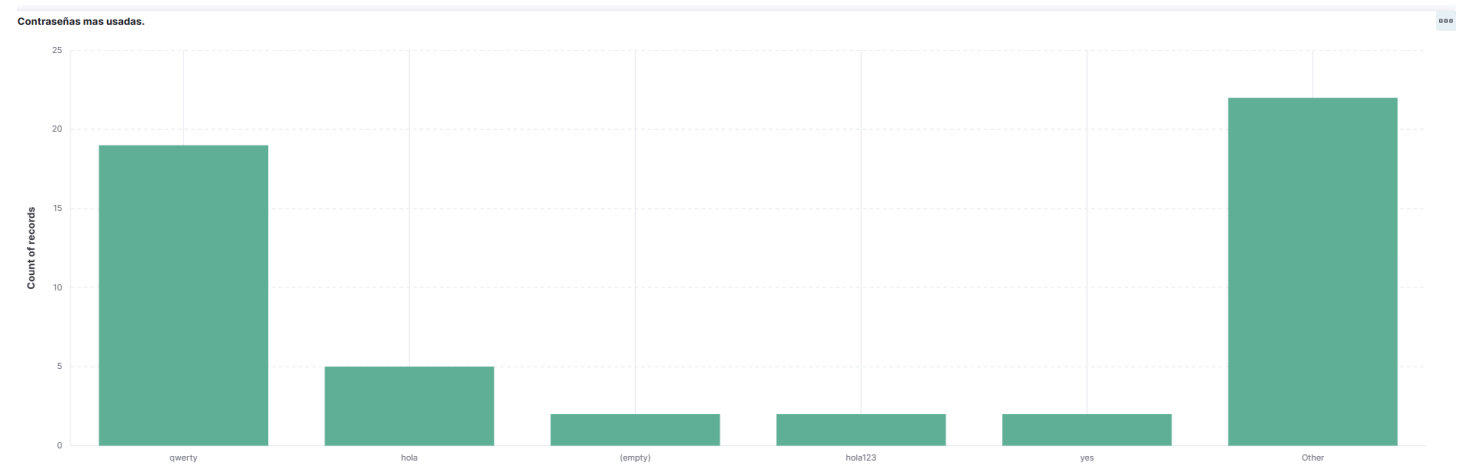
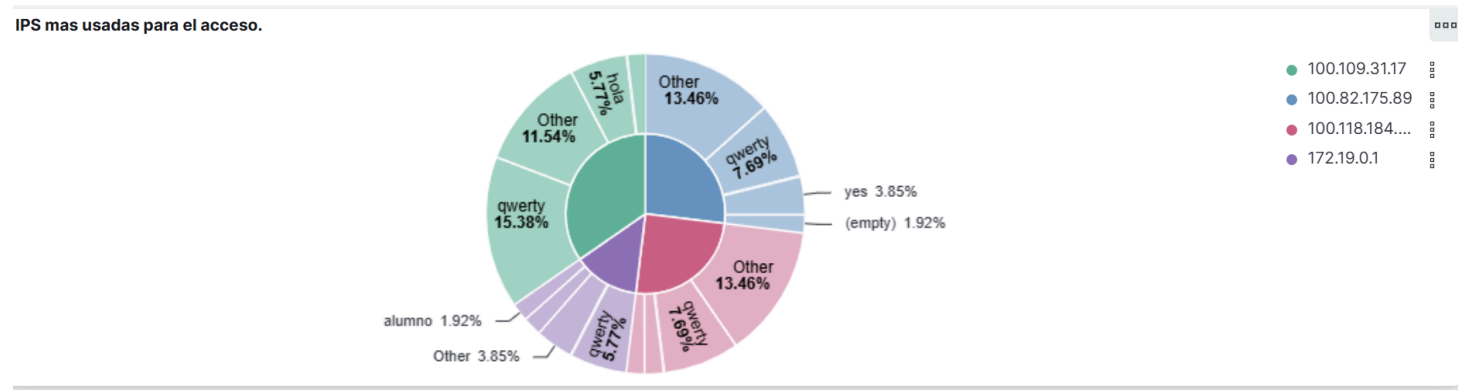


18.11. Consolidación del Dashboard de seguridad en Kibana.

Una vez definida la configuración, procedemos a integrar las diferentes vistas en un panel de control unificado. En la captura se observa el resultado final de este proceso, donde hemos combinado dos tipos de análisis fundamentales para la monitorización del Honeypot. A la izquierda, un gráfico circular que nos permite visualizar la proporción de ataques según la dirección IP de origen y contraseñas de acceso usadas por esa IP, así podremos determinar si realmente se trata de un ataque.

A la derecha, hemos incorporado un gráfico de barras que detalla las contraseñas más usadas en intentos de acceso. Esta información es de gran valor para el análisis forense ya que nos permite conocer qué tipos de ataque podrían estar utilizando bots automatizados, por ejemplo mediante librerías de contraseñas comunes y qué nivel tienen las amenazas. Tener ambos

gráficos en un mismo Dashboard nos permite supervisar el estado de nuestro servidor de un solo vistazo transformando miles de logs en una herramienta totalmente visual y sencilla de interpretar.



18.12. Almacenamiento del Dashboard de análisis.

Para finalizar la fase de monitorización, procedemos a guardar el conjunto de gráficos creados con un nombre descriptivo. En la imagen se observa el formulario de guardado donde hemos guardado el panel, así nos aseguramos que toda la configuración con los filtros, ejes y tipos de gráficos no se pierda y esté disponible cada vez que accedemos a Kibana.

Además, el sistema nos permite activar la opción de "Store time with dashboard", que asegura que al abrirlo de nuevo se mantenga la misma hora que estábamos analizando.

×

Save dashboard

Title

Tablas de Valores del Honeypot

Description

Optional

Tags

Optional

☒ Store time with dashboard

This changes the time filter to the currently selected time each time this dashboard is loaded.

Cancel

Save

19. Conclusiones.

La configuración de **Edge Computing** confirma que es totalmente factible unir servicios de inteligencia artificial y ciberseguridad avanzada en un entorno local. Al integrar **Dolphin**, se ha logrado que la IA funcione como un apoyo estratégico que mediante instrucciones específicas es capaz de interpretar logs del servidor y sugerir soluciones técnicas en tiempo real. Lo más importante de este enfoque es que garantiza la **soberanía de nuestros datos**: toda la información se queda en nuestra propias manos, eliminando los riesgos de privacidad y la dependencia de servicios externos en la nube.

En cuanto a la seguridad, el servidor ha logrado completar una defensa proactiva. Hemos combinado un entorno de engaño con **Honeypot Cowrie** para atraer y desviar ataques con el análisis detallado que ofrece el **Stack ELK**. Esto nos permite convertir los intentos de intrusión en datos legibles para reforzar la seguridad del servidor de forma constante. Además, el uso de **Docker** para separar los servicios y de **Ansible** para gestionar la configuración nos asegura que la arquitectura sea profesional y fácil de replicar, algo fundamental en un entorno de administración de sistemas.

20. Posibles aplicaciones y mejoras.

A pesar de haberse alcanzado los objetivos definidos, la infraestructura diseñada es totalmente escalable y ofrece diferentes posibilidades de ampliación sin necesidad de rediseñar la arquitectura base. Entre las vías para potenciar sus capacidades se encuentran:

- **Protección avanzada del Dashboard:** Una de las mejoras prioritarias es añadir una capa de seguridad adicional mediante un sistema de acceso con usuario y contraseña para entrar a los paneles de control de **Kibana**. Esto nos aseguraría que las gráficas y mapas donde vemos los ataques recibidos sean totalmente privados con solamente acceso autorizado.
- **Asistente técnico especializado:** Otra opción sería entrenar a la IA local con una base de datos que guarde todos los datos de este proyecto y los logs de errores que han ido surgiendo. De esta manera, tendríamos a alguien que nos ayude a solucionar problemas técnicos, basándose en lo que ha ocurrido en nuestro servidor.
- **Defensa autónoma:** Se podría configurar el sistema para que al detectar que alguien intenta entrar múltiples veces, el servidor tome decisiones por sí solo como bloquear el acceso o aislar ciertas partes críticas de la red de forma inmediata para evitar riesgos sin intervención manual.
- **Conexiones web protegidas:** Otra mejora interesante sería automatizar la creación de certificados para que todas las aplicaciones que abramos en el navegador funcionen siempre con una conexión cifrada y segura, garantizando que nadie pueda interceptar nuestros datos.

21. Anexos del Proyecto.

En los siguientes anexos se recogen las configuraciones completas y evidencias generadas durante el despliegue de los distintos servicios del sistema. Estos anexos complementan la memoria principal y permiten consultar información técnica detallada de los servicios desplegados.

[ANEXO I – Fail2Ban](#)

Se incluye la configuración completa del servicio Fail2Ban, concretamente el fichero jail.local, donde se definen las políticas de bloqueo, tiempos de ban y reglas aplicadas al servicio SSH.

[ANEXO II – OpenWebUI](#)

Se incluye la configuración del despliegue del contenedor correspondiente a la interfaz web utilizada para la interacción con el modelo de inteligencia artificial.

[ANEXO III – Nginx y SSL](#)

Se adjuntan las configuraciones de los sites-enabled de Nginx utilizadas en el despliegue del sistema, así como la referencia a los certificados SSL empleados.

[ANEXO IV – Elasticsearch, Kibana y Filebeat.](#)

En este anexo se recogen las configuraciones de los servicios Elasticsearch, Kibana y Filebeat desplegados en el proyecto, incluyendo tanto archivos de configuración como contenedores utilizados.

[ANEXO V – Honeybot](#)

Se incluye la configuración del honeypot desplegado mediante Cowrie (cowrie.cfg), que simula un servicio SSH falso. Además, se adjunta el script de análisis en Python, encargado de procesar los eventos mediante inteligencia artificial y generar notificaciones de alertas de acceso hacia un canal de Discord.

[ANEXO VI – Logs del Honeypot \(Cowrie\)](#)

En este anexo se recogen los registros generados por el honeypot desplegado mediante Cowrie. Estos logs reflejan los intentos de acceso al servicio SSH falso, incluyendo credenciales utilizadas, comandos ejecutados por los atacantes y direcciones IP de origen.

Dicha información ha sido posteriormente analizada para detectar patrones de ataque y generar alertas automatizadas mediante el sistema de notificaciones integrado con Discord y el módulo de análisis basado en inteligencia artificial.

[ANEXO VII – Uso de Inteligencia Artificial](#)

Durante el desarrollo del proyecto se han utilizado herramientas de inteligencia artificial como apoyo técnico, concretamente **Chat GPT** y **Claude**.

Estas herramientas han sido utilizadas en distintas fases del proyecto:

- Generación y apoyo en el desarrollo del **script en Python** para el sistema de alertas mediante Discord Webhooks, esto sería casi imposible de realizarlo debido a que no tenemos los conocimientos avanzados ni generales de este lenguaje de programación.
- Asistencia en la **configuración y despliegue del stack ELK (Elasticsearch, Kibana y Filebeat)**, debido a su complejidad de despliegue y enlaces entre estas distintas tecnologías.
- Resolución de dudas técnicas relacionadas con Docker, redes, seguridad y análisis de logs ya que ciertos parámetros de las tecnologías son difíciles de interpretar debido a la falta de conocimientos sobre ellos.

Todas las configuraciones, scripts y soluciones aplicadas han sido probadas, modificadas y validadas manualmente, garantizando su correcto funcionamiento dentro del entorno real del proyecto, para esto también adjuntamos una carpeta con todas las configuraciones de nuestro proyecto como evidencia de la edición de los mismos y comprobación de su funcionamiento.

▣ SOVEREIGN-EDGE IA, Anexos

El uso de estas herramientas se ha realizado como complemento al aprendizaje técnico, permitiendo mejorar la eficiencia sin sustituir los conocimientos adquiridos durante el desarrollo del proyecto.

22. Bibliografía.

- Docker Inc. (2026). *Docker Documentation*. <https://docs.docker.com> ,
<https://docs.docker.com/manuals/>
- Canonical Ltd. (2026). *Ubuntu Server Documentation*. <https://ubuntu.com/server/docs>
- Red Hat. (2026). *Ansible Documentation*. <https://docs.ansible.com>
- Tailscale Inc. (2026). *Tailscale Documentation*. <https://tailscale.com/docs>
- Cowrie Project. (2026). *Cowrie Honeypot*. <https://github.com/cowrie/cowrie>
- Fail2Ban Developers. (2026). *Fail2Ban Documentation*. <https://www.fail2ban.org>
- Elastic. (2026). *Elastic Stack (Elasticsearch, Kibana, Filebeat)*.
<https://www.elastic.co/docs/solutions/observability>
[,https://www.elastic.co/docs/reference/beats/filebeat](https://www.elastic.co/docs/reference/beats/filebeat)
- Nginx Inc. (2026). *NGINX Documentation*. <https://nginx.org/en/docs>
- Ollama. (2026). *Ollama Documentation*. <https://ollama.com/library/llama3.1> ,
<https://ollama.com/library/dolphin3>
- Meta AI. (2026). *Llama Models*. <https://ai.meta.com/llama>
- OpenAI. (2024). *Chat GPT*. <https://chat.openai.com>
- Anthropic. (2024). *Claude*. <https://www.anthropic.com>
- Discord Webhooks (2026). [Webhooks](#).